Bölüm 13: Karar Alma

Algoritmik Oyun Kuramı

Beklenen fayda | Risk tutumları | Belirsizlik ölçütleri | Bayes kuralı | Karar ağaçları | Bilginin değeri | Beklenti kuramı | Ortak mallar trajedisi

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

BEKLENEN FAYDA

Öğrenme Çıktıları

Beklenen fayda ve risk

Bir piyangonun beklenen değerini ve faydasını hesaplamak; kesinlik eşdeğeri ve risk primiyle risk tutumunu yorumlamak; olasılıklar bilinmezken maksimin, Laplace ve Savage ölçütlerini uygulamak.

Bayes kuralı

Önsel inancı test sonucuyla güncellemek; taban oranı ihmalini sayısal örnekte göstermek.

Karar ağaçları

Geriye doğru çıkarımla en iyi stratejiyi bulmak; mükemmel ve örnek bilginin değerini hesaplamak.

Davranışsal ve kolektif karar

Beklenti kuramı ve kayıptan kaçınmayı açıklamak; ortak mallar trajedisini Nash dengesiyle hesaplamak.

BEKLENEN FAYDA

Karar Alma: Türler ve Bu Bölümün Yeri

Karar alma, seçenekler arasından sonuçlarını değerlendirerek **seçim yapma** sürecidir. Kararı etkileyen şeye göre üç tür:

Tür	Sonucu belirleyen	Araç	Örnek
Bireysel, risk altında	Doğa / şans (olasılıklar bilinir ya da tahmin edilir)	Beklenen fayda, Bayes, karar ağacı	Yatırım, sigorta, tıbbi test
Stratejik	Diğer rasyonel aktörlerin seçimleri	En iyi tepki, Nash dengesi	Fiyat rekabeti, pazarlık, spor
Kolektif	Grubun toplu seçimi ve etkileşimi	Oylama, ortak mallar	Politika, şirket stratejisi

Nash dengesi karşılıklı stratejik bağımlılığı çözer ama sosyal açıdan en iyi sonucu garanti etmez.

BEKLENEN FAYDA

Rasyonel Karar: Beklenen Değer mi, Beklenen Fayda mı?

- **Piyango** (lottery): olasılıklı sonuçlar kümesi. A = %50 olasılıkla 900 TL, %50 olasılıkla 0 TL.
- **Beklenen değer**: $E[X] = \Sigma\, p_i\, x_i$. $E[A] = ½\cdot900 + ½\cdot0 =$ **450** TL > kesin B = 400 TL.
- **Beklenen fayda** (expected utility; von Neumann–Morgenstern): $E[u] = \Sigma\, p_i\, u(x_i)$. Rasyonel karar verici beklenen *faydayı* en büyükler.
- $u(x) = \sqrt{x}$ ile: $E[u(A)] = ½\cdot30 + ½\cdot0 =$ **15**; $u(B) = \sqrt{400} =$ **20** ⇒ B seçilir.



Turuncu: her ölçütün seçtiği. Beklenen değer A'yı, beklenen fayda (√x) B'yi seçer.

Beklenen parasal değer ile beklenen fayda **aynı kararı vermeyebilir**; aradaki farkı fayda fonksiyonunun eğriliği, yani risk tutumu belirler.

St. Petersburg paradoksu: beklenen değeri sonsuz bir oyuna kimse büyük para ödemez; Bernoulli (1738) bu yüzden faydayı önerdi.

Beklenen Faydanın Temeli ve Allais Paradoksu

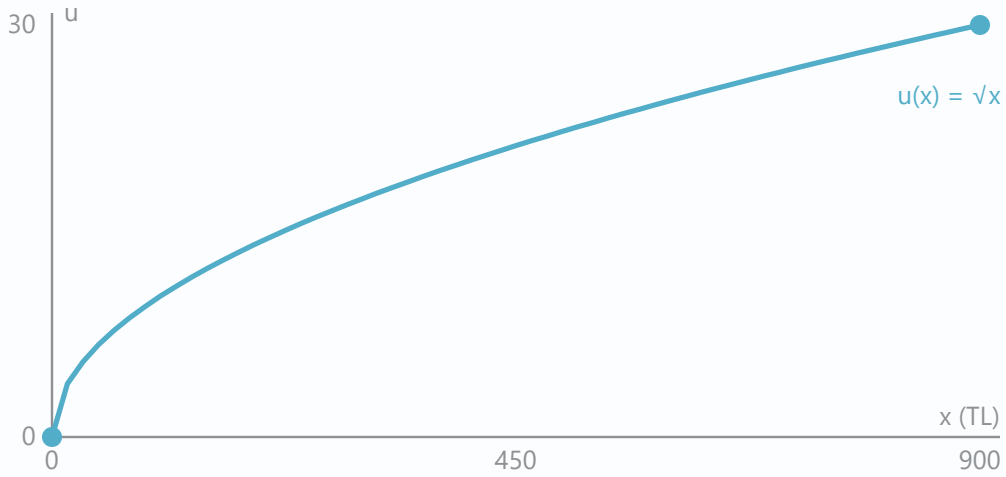
- **von Neumann–Morgenstern**: tercihler *tamlık*, *geçişlilik*, *süreklilik* ve *bağımsızlık* aksiyomlarını sağlıyorsa bir u vardır ve karar verici beklenen faydayı en büyükler.
- **Bağımsızlık**: iki piyangoya aynı ortak parçayı (aynı olasılıkla) eklemek sıralamayı değiştirmez.
- **Allais (1953)**: çoğu kişi 1. seçimde A'yı, 2. seçimde D'yi seçer.

	Seçenek	1 milyon	5 milyon	0
1. seçim	A	%100	—	—
	B	%89	%10	%1
2. seçim	C	%11	—	%89
	D	—	%10	%90

A ve B'deki ortak %89 “1 milyon” parçası %89 “0” ile değiştirilince C ve D elde edilir.

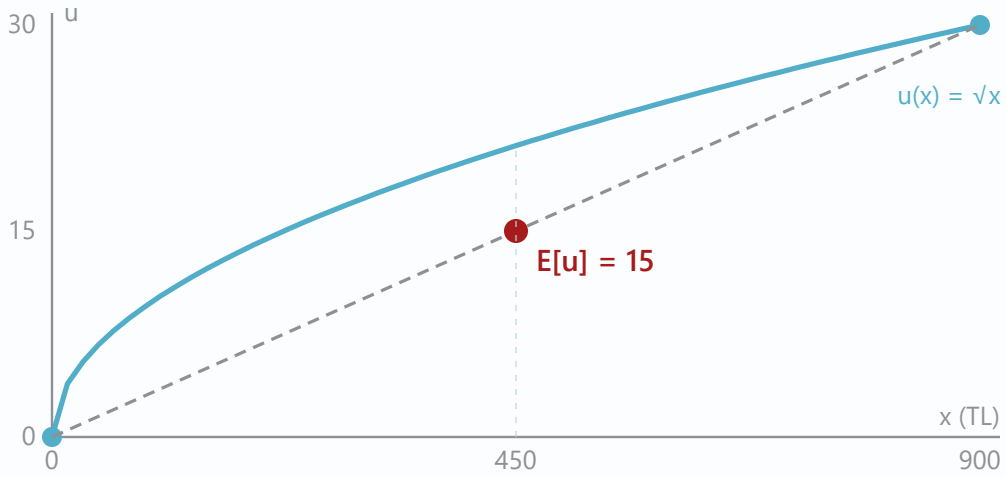
Her u için $E[u(A)] - E[u(B)] = 0{,}11\cdot u(1) - 0{,}10\cdot u(5) - 0{,}01\cdot u(0) = E[u(C)] - E[u(D)]$. Yani $A \succ B$ ise $C \succ D$ olmalı; “A ve D” seçimi **hiçbir** fayda fonksiyonuyla açıklanamaz. Kesinliğe verilen fazla ağırlık (kesinlik etkisi) beklenti kuramının çıkış noktalarından biridir.

Kesinlik Eşdeğeri ve Risk Primi Adım Adım



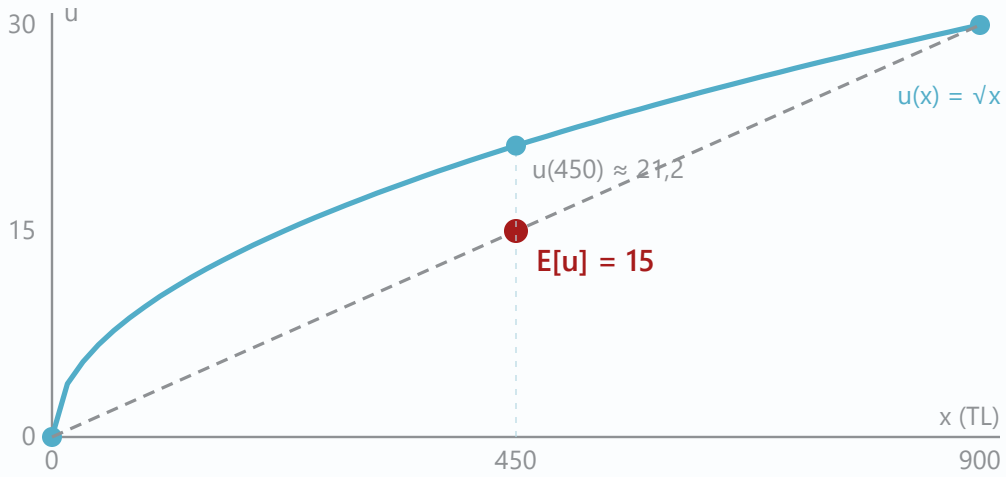
İçbükey fayda u(x) = √x: her ek lira bir öncekinden daha az fayda katar (azalan marjinal fayda). Uçlar: u(0) = 0, u(900) = 30.

Kesinlik Eşdeğeri ve Risk Primi Adım Adım



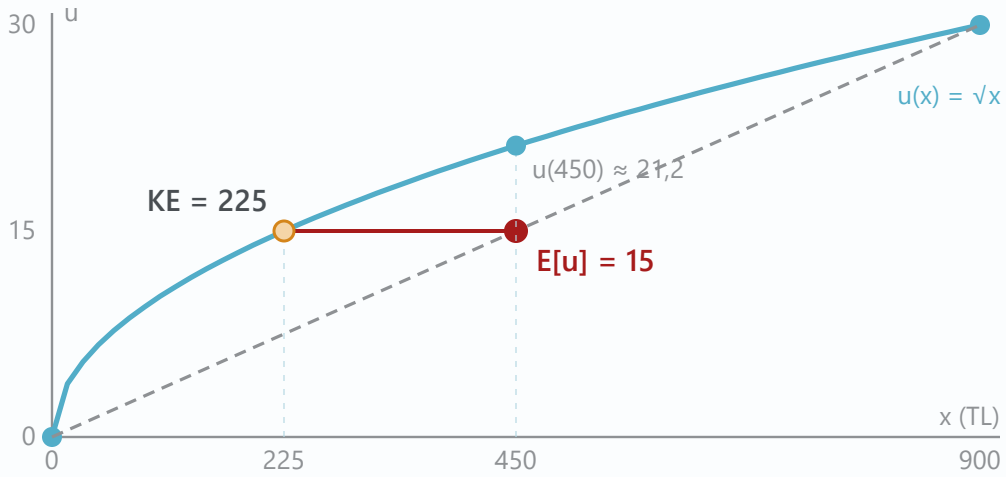
Piyango A'nın beklenen faydası iki ucu birleştiren kirişin ortasıdır: E[u] = ½·0 + ½·30 = **15** (x = 450 hizasında).

Kesinlik Eşdeğeri ve Risk Primi Adım Adım



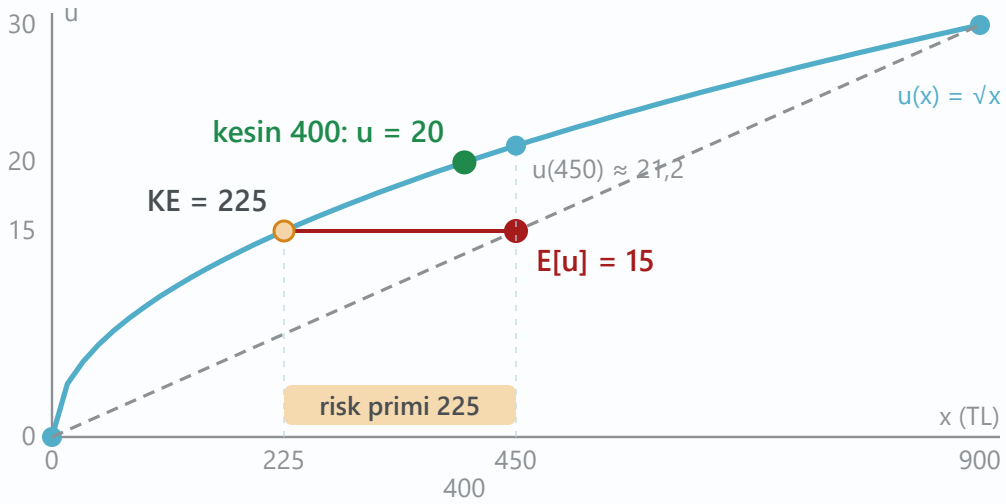
Aynı 450 TL kesin verilse fayda u(450) ≈ 21,2 > 15. Kiriş eğrinin altında kalır: içbükey fonksiyonda E[u(X)] < u(E[X]) (Jensen).

Kesinlik Eşdeğeri ve Risk Primi Adım Adım



Kesinlik eşdeğeri (KE): faydası 15 olan kesin tutar. √x = 15 ⇒ **KE = 225 TL**. Bu kişi piyangoyu 225 TL'ye satmaya razıdır.

Kesinlik Eşdeğeri ve Risk Primi Adım Adım



Risk primi = E[X] − KE = 450 − 225 = **225 TL**. Kesin 400 TL'nin faydası 20 > 15 ⇒ riskten kaçınan kişi **B**'yi seçer.

RİSK TUTUMLARI

Üç Risk Tutumu: Aynı Piyango, Farklı Kararlar

Tutum	Fayda	E[u(A)]	u(400)	Seçim	KE (TL)	Risk primi
Riskten kaçınan	u(x) = √x (içbükey)	15	20	B	225	225
Risk nötr	u(x) = x (doğrusal)	450	400	A	450	0
Risk seven	u(x) = x^2/900 (dışbükey)	450	177,8	A	636,4	−186,4

İçbükey → kaçınan

KE < E[X], prim > 0: riski üstlenmek için ödül ister. İnsanların çoğu, büyük tutarlarda.

Doğrusal → nötr

KE = E[X]: yalnızca beklenen değere bakar. Büyük şirketler, küçük tutarlar.

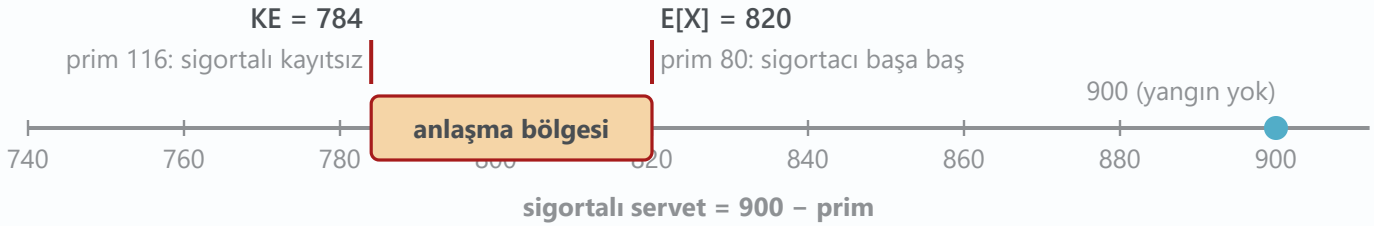
Dışbükey → seven

KE > E[X], prim < 0: risk için para öder. Piyango bileti alan kişi.

Piyango A: %50 900 TL, %50 0 TL (E[X] = 450). B: kesin 400 TL. KE = u^{-1}(E[u(A)]), risk primi = E[X] − KE.

Uygulama: Sigorta Neden Var?

- Servet 900 (bin TL); %10 olasılıkla 800'lük kayıp (yangın). Fayda $u(x) = \sqrt{x}$.
- Sigortasız: $E[u] = 0{,}9 \cdot \sqrt{900} + 0{,}1 \cdot \sqrt{100} = 27 + 1 = \mathbf{28}$.
- Kesinlik eşdeğeri: $28^2 = \mathbf{784}$. Kişi riskli 900 yerine kesin 784'e razı.
- En fazla ödenecek prim = 900 − 784 = **116**; beklenen kayıp yalnızca 0,1·800 = 80.



Sigortalı: prim ≤ 116 ödeyip riskten kurtulmak faydasını artırır. Sigortacı: çok sayıda bağımsız poliçeyle risk nötre yakındır; 80'den fazla her prim kârdır.

80 ile 116 arasındaki her prim **iki taraf için de kazançlıdır**: risk, ona en az katlanan tarafa aktarılır.

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü
Büyük yatırım	200	50	−120
Küçük yatırım	90	40	−20
Yatırım yok	0	0	0

Olasılıklar **bilinmiyor** (belirsizlik). Satırlar eylemler, sütunlar durumlar; her ölçüt farklı bir "karamsarlık" derecesi taşır.

Adım 1 / 8

```
1  def olcutler(U):
▶      A, m = range(len(U)), len(U[0])
3      mm = max(A, key=lambda i: min(U[i]))      # maximin
4      mx = max(A, key=lambda i: max(U[i]))      # maximax
5      lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
6      enb = [max(r[j] for r in U) for j in range(m)]
7      pis = [max(enb[j] - r[j] for j in range(m))
8             for r in U]            # en büyük pişmanlık
9      sv = pis.index(min(pis))                 # Savage
10     return mm, mx, lp, sv
```

DEĞİŞKENLER

`U = 3×3 tablo` `m = 3`

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü	min
Büyük yatırım	200	50	−120	−120
Küçük yatırım	90	40	−20	−20
Yatırım yok	0	0	0	0

Maksimin (maximin; Wald, en kötüyü düşün): satır minimumları −120, −20, 0 → en büyüğü 0: **Yatırım yok**.

Adım 2 / 8

```
1  def olcutler(U):
2      A, m = range(len(U)), len(U[0])
▶      mm = max(A, key=lambda i: min(U[i]))      # maximin
4      mx = max(A, key=lambda i: max(U[i]))      # maximax
5      lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
6      enb = [max(r[j] for r in U) for j in range(m)]
7      pis = [max(enb[j] - r[j] for j in range(m))
8             for r in U]            # en büyük pişmanlık
9      sv = pis.index(min(pis))                  # Savage
10     return mm, mx, lp, sv
```

DEĞİŞKENLER

`U = 3×3 tablo` `mm = 2 (Yok)`

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü	min	max
Büyük yatırım	200	50	−120	−120	200
Küçük yatırım	90	40	−20	−20	90
Yatırım yok	0	0	0	0	0

Maksimaks (maximax; iyimser): satır maksimumları 200, 90, 0 → **Büyük yatırım**.

Adım 3 / 8

```python
def olcutler(U):
    A, m = range(len(U)), len(U[0])
    mm = max(A, key=lambda i: min(U[i]))      # maximin
    mx = max(A, key=lambda i: max(U[i]))      # maximax
    lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
    enb = [max(r[j] for r in U) for j in range(m)]
    pis = [max(enb[j] - r[j] for j in range(m))
           for r in U]            # en büyük pişmanlık
    sv = pis.index(min(pis))                  # Savage
    return mm, mx, lp, sv
```

DEĞİŞKENLER

`U = 3×3 tablo` `mm = 2` `mx = 0 (Büyük)`

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü	min	max	ort.
Büyük yatırım	200	50	−120	−120	200	43,3
Küçük yatırım	90	40	−20	−20	90	36,7
Yatırım yok	0	0	0	0	0	0

Laplace (durumlar eşit olası say): ortalamalar 43,3, 36,7, 0,0 → **Büyük yatırım**.

Adım 4 / 8

```
1  def olcutler(U):
2      A, m = range(len(U)), len(U[0])
3      mm = max(A, key=lambda i: min(U[i]))      # maximin
4      mx = max(A, key=lambda i: max(U[i]))      # maximax
▶      lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
6      enb = [max(r[j] for r in U) for j in range(m)]
7      pis = [max(enb[j] - r[j] for j in range(m))
8             for r in U]            # en büyük pişmanlık
9      sv = pis.index(min(pis))                  # Savage
10     return mm, mx, lp, sv
```

DEĞİŞKENLER

`U = 3×3 tablo` `mm = 2` `mx = 0` `lp = 0 (Büyük)`

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü	min	max	ort.
Büyük yatırım	200	50	−120	−120	200	43,3
Küçük yatırım	90	40	−20	−20	90	36,7
Yatırım yok	0	0	0	0	0	0
sütun en iyisi:	200	50	0			

Her durumda ulaşılabilecek en iyi sonuç: enb = [200, 50, 0]. Pişmanlık = enb − elde edilen.

Adım 5 / 8

```
1  def olcutler(U):
2      A, m = range(len(U)), len(U[0])
3      mm = max(A, key=lambda i: min(U[i]))      # maximin
4      mx = max(A, key=lambda i: max(U[i]))      # maximax
5      lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
▶      enb = [max(r[j] for r in U) for j in range(m)]
7      pis = [max(enb[j] - r[j] for j in range(m))
8             for r in U]            # en büyük pişmanlık
9      sv = pis.index(min(pis))                  # Savage
10     return mm, mx, lp, sv
```

DEĞİŞKENLER

`U = 3×3 tablo` `mm = 2` `mx = 0` `lp = 0` `enb = [200, 50, 0]`

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü	min	max	ort.	maks. pişm.
Büyük yatırım	200	50	−120	−120	200	43,3	120
Küçük yatırım	90	40	−20	−20	90	36,7	110
Yatırım yok	0	0	0	0	0	0	200
sütun en iyisi:	200	50	0				

Satır başına en büyük pişmanlık: Büyük (kötü durumda 0 − (−120)) = 120, Küçük (iyide 200 − 90) = 110, Yok (iyide 200) = 200.

Adım 6 / 8

```
1  def olcutler(U):
2      A, m = range(len(U)), len(U[0])
3      mm = max(A, key=lambda i: min(U[i]))      # maximin
4      mx = max(A, key=lambda i: max(U[i]))      # maximax
5      lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
6      enb = [max(r[j] for r in U) for j in range(m)]
▶      pis = [max(enb[j] - r[j] for j in range(m))
▶             for r in U]            # en büyük pişmanlık
9      sv = pis.index(min(pis))                  # Savage
10     return mm, mx, lp, sv
```

DEĞİŞKENLER

`U = 3×3 tablo` `mm = 2` `mx = 0` `lp = 0` `enb = [200, 50, 0]` `pis = [120, 110, 200]`

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü	min	max	ort.	maks. pişm.
Büyük yatırım	200	50	−120	−120	200	43,3	120
Küçük yatırım	90	40	−20	−20	90	36,7	110
Yatırım yok	0	0	0	0	0	0	200
sütun en iyisi:	200	50	0				

Savage (minimaks pişmanlık): en küçük maksimum pişmanlık 110 → **Küçük yatırım**.

Adım 7 / 8

```
1  def olcutler(U):
2      A, m = range(len(U)), len(U[0])
3      mm = max(A, key=lambda i: min(U[i]))      # maximin
4      mx = max(A, key=lambda i: max(U[i]))      # maximax
5      lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
6      enb = [max(r[j] for r in U) for j in range(m)]
7      pis = [max(enb[j] - r[j] for j in range(m))
8             for r in U]            # en büyük pişmanlık
▶      sv = pis.index(min(pis))                 # Savage
10     return mm, mx, lp, sv
```

DEĞİŞKENLER

U = 3×3 tablo | mm = 2 | mx = 0 | lp = 0 | pis = [120, 110, 200] | sv = 1 (Küçük)

Olasılıklar Bilinmezse: Belirsizlik Altında Karar

eylem \ durum	İyi	Orta	Kötü	min	max	ort.	maks. pişm.
Büyük yatırım	200	50	−120	−120	200	43,3	120
Küçük yatırım	90	40	−20	−20	90	36,7	110
Yatırım yok	0	0	0	0	0	0	200
sütun en iyisi:	200	50	0				

return (2, 0, 0, 1): aynı tabloda üç farklı karar! Olasılıklar tahmin edilebiliyorsa beklenen faydaya dönülür; edilemiyorsa ölçüt seçimi karar vericinin tutumunu yansıtır.

Adım 8 / 8

```
1 def olcutler(U):
2     A, m = range(len(U)), len(U[0])
3     mm = max(A, key=lambda i: min(U[i]))      # maximin
4     mx = max(A, key=lambda i: max(U[i]))      # maximax
5     lp = max(A, key=lambda i: sum(U[i]) / m)  # Laplace
6     enb = [max(r[j] for r in U) for j in range(m)]
7     pis = [max(enb[j] - r[j] for j in range(m))
8            for r in U]            # en büyük pişmanlık
9     sv = pis.index(min(pis))                  # Savage
▶     return mm, mx, lp, sv
```

DEĞİŞKENLER

`sonuç = (2, 0, 0, 1)`

BAYES KURALI

Bayes Kuralı (Bayes' Rule)

$$P(H \mid E) = P(E \mid H) \cdot P(H) / P(E), \quad P(E) = P(E \mid H) \cdot P(H) + P(E \mid \neg H) \cdot P(\neg H)$$

Önsel (prior) P(H)
Kanıttan önceki inanç: hastalığın toplumdaki yaygınlığı.

Olabilirlik (likelihood) P(E | H)
Hipotez doğruysa kanıtı görme olasılığı: testin duyarlılığı.

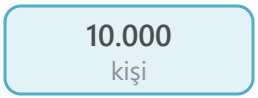
Kanıt P(E)
Kanıtın toplam olasılığı: tüm pozitiflerin oranı (hasta ve sağlıklı).

Sonsal (posterior) P(H | E)
Kanıttan sonraki inanç; sonraki kanıt için yeni önsel olur.

Oyun kuramında eksik bilgili oyunlarda (Bayesçi oyunlar) oyuncular rakibin tipi hakkındaki inançlarını aynı kuralla günceller.

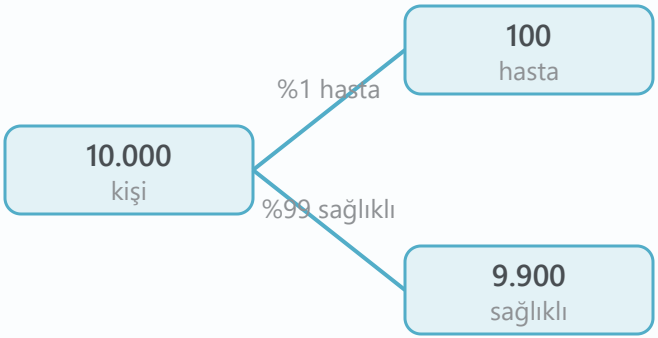
BAYES KURALI

Tıbbi Test: Pozitif Sonuç Ne Anlama Gelir?



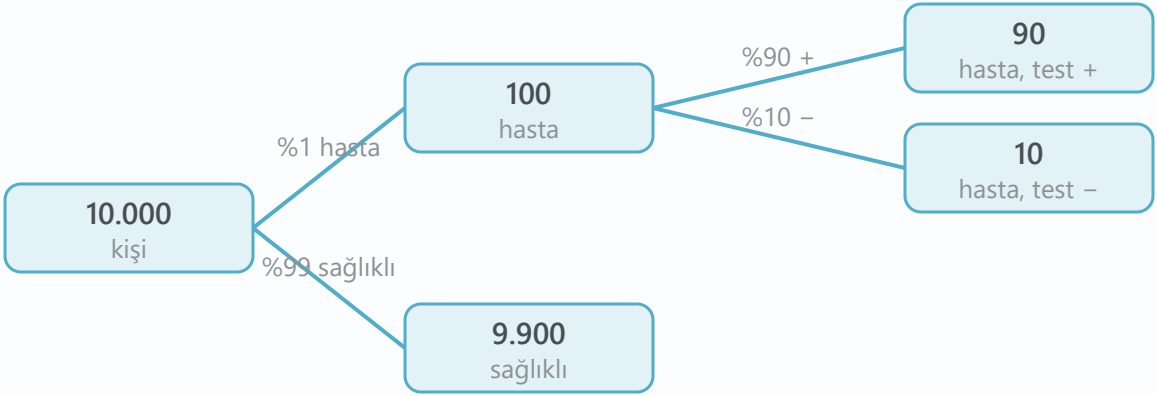
10.000 kişilik bir topluluk düşünelim. Olasılıklar yerine **doğal sıklıklar** (kişi sayıları) kullanmak Bayes hesabını sezgisel kılar.

Tıbbi Test: Pozitif Sonuç Ne Anlama Gelir?



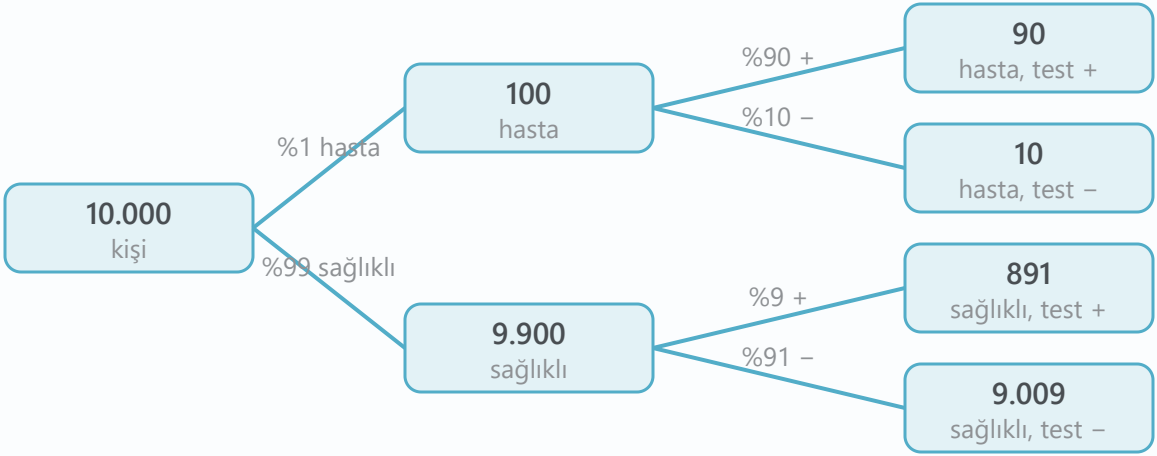
Önsel (prior): yaygınlık %1 ⇒ 100 hasta, 9.900 sağlıklı.

Tıbbi Test: Pozitif Sonuç Ne Anlama Gelir?



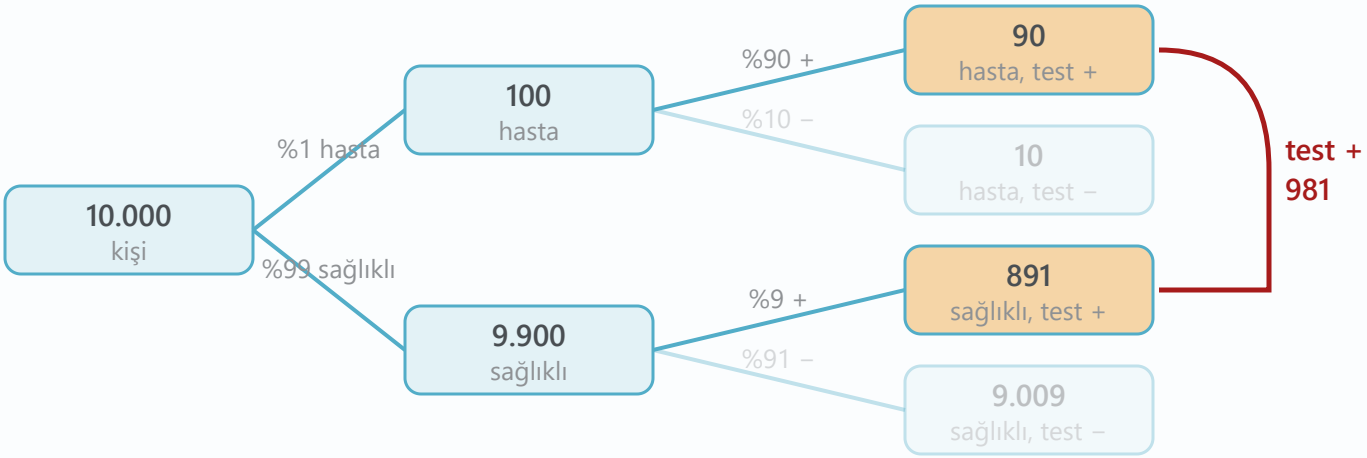
Duyarlılık (sensitivity) %90: 100 hastanın 90'ı pozitif, 10'u negatif çıkar.

Tıbbi Test: Pozitif Sonuç Ne Anlama Gelir?



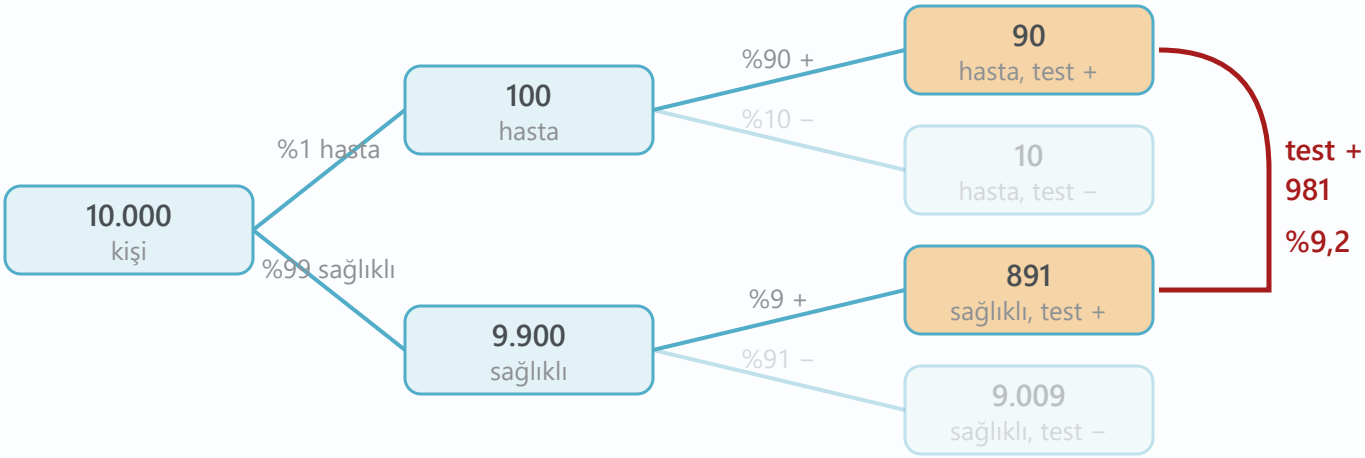
Yanlış pozitif oranı %9: 9.900 sağlıklının 891'i de pozitif çıkar! Sağlıklılar çok kalabalık olduğu için bu sayı büyüktür.

Tıbbi Test: Pozitif Sonuç Ne Anlama Gelir?



Test pozitif çıkan herkes: 90 + 891 = **981** kişi. Bunlardan yalnızca 90'ı gerçekten hasta.

Tıbbi Test: Pozitif Sonuç Ne Anlama Gelir?



Sonsal: P(hasta | +) = 90 / 981 = **%9,2**. Test %90 duyarlı olsa da pozitif sonuç çoğunlukla yanlış alarmdır (taban oranı ihmali).

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



Çağrı: `bayes(0.01, 0.9, 0.09, ["+", "+", "−"])`. p önsel olasılıkla (%1) başlar; üç test sonucu sırayla işlenecek.

Adım 1 / 14

```
▶ def bayes(p, duyarlilik, yp, testler):
2   for sonuc in testler:
3     if sonuc == "+":
4       pay = duyarlilik * p
5       payda = pay + yp * (1 - p)
6     else:
7       pay = (1 - duyarlilik) * p
8       payda = pay + (1 - yp) * (1 - p)
9     p = pay / payda
10    print(sonuc, round(p, 4))
11  return p
```

DEĞİŞKENLER

`p = 0.0100` `duyarlilik = 0.9` `yp = 0.09` `testler = [+, +, −]`

ÇIKTI

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



1. test sonucu: **+**. Şu anki inanç p = 0.0100 bu testin önseli olur.

Adım 2 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
▶    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

DEĞİŞKENLER

`p = 0.0100` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +`

ÇIKTI

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



sonuc == "+" ? → **Doğru**: pozitif sonucun olabilirlikleri kullanılır.

Adım 3 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
▶      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

DEĞİŞKENLER

`p = 0.0100` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +`

ÇIKTI

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



pay = P(+|hasta)·p = 0.9 × 0.0100 = 0.0090; payda = pay + P(+|sağlıklı)·(1 − p) = 0.0981 (toplam pozitif olasılığı).


```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
3      if sonuc == "+":
▶        pay = duyarlilik * p
▶        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

DEĞİŞKENLER

`p = 0.0100` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +` `pay = 0.0090` `payda = 0.0981`

ÇIKTI

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



p = pay / payda = **0.0917**; print ile yazdırılır. Eşiğin (%13) altında: tedaviye başlamak için yetersiz.

Adım 5 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
▶      p = pay / payda
▶      print(sonuc, round(p, 4))
11   return p
```

DEĞİŞKENLER

`p = 0.0917` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +` `pay = 0.0090` `payda = 0.0981`

ÇIKTI

```
+ 0.0917
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



2. test sonucu: **+**. Şu anki inanç p = 0.0917 bu testin önseli olur.

Adım 6 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
▶    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

DEĞİŞKENLER

`p = 0.0917` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +`

ÇIKTI

```
+ 0.0917
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



sonuc == "+" ? → **Doğru**: pozitif sonucun olabilirlikleri kullanılır.


```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
▶      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

DEĞİŞKENLER

`p = 0.0917` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +`

ÇIKTI

```
+ 0.0917
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



pay = P(+|hasta)·p = 0.9 × 0.0917 = 0.0826; payda = pay + P(+|sağlıklı)·(1 − p) = 0.1643 (toplam pozitif olasılığı).


```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
3      if sonuc == "+":
▶        pay = duyarlilik * p
▶        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

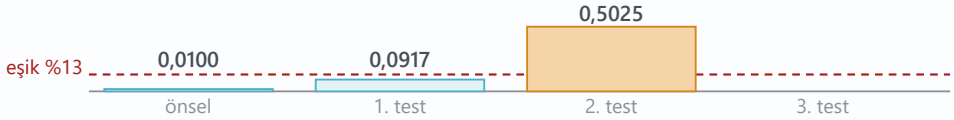
DEĞİŞKENLER

`p = 0.0917` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +` `pay = 0.0826` `payda = 0.1643`

ÇIKTI

```
+ 0.0917
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



p = pay / payda = **0.5025**; print ile yazdırılır. Eşiğin üstüne çıktı: iki bağımsız pozitif, olasılığı ~%50'ye taşıdı.


```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
▶      p = pay / payda
▶      print(sonuc, round(p, 4))
11   return p
```

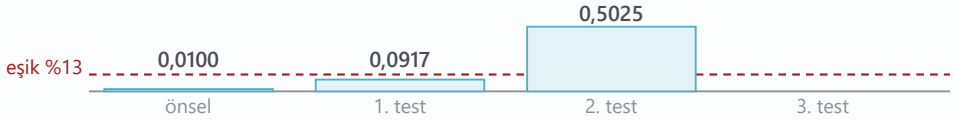
DEĞİŞKENLER

`p = 0.5025` `duyarlilik = 0.9` `yp = 0.09` `sonuc = +` `pay = 0.0826` `payda = 0.1643`

ÇIKTI

```
+ 0.0917
+ 0.5025
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



3. test sonucu: **-**. Şu anki inanç p = 0.5025 bu testin önseli olur.


```
1  def bayes(p, duyarlilik, yp, testler):
▶    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

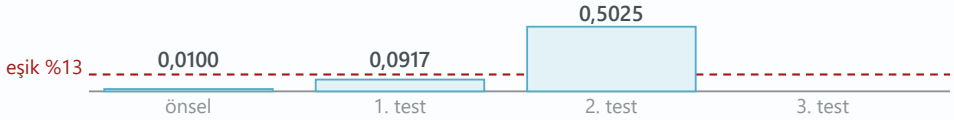
DEĞİŞKENLER

`p = 0.5025` `duyarlilik = 0.9` `yp = 0.09` `sonuc = -`

ÇIKTI

```
+ 0.0917
+ 0.5025
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



sonuc == "+" ? → **Yanlış**: negatif sonucun olabilirlikleri kullanılır.

Adım 11 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
▶      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

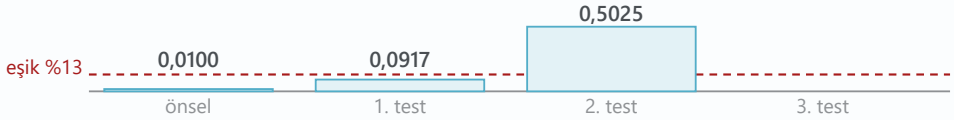
DEĞİŞKENLER

`p = 0.5025` `duyarlilik = 0.9` `yp = 0.09` `sonuc = -`

ÇIKTI

```
+ 0.0917
+ 0.5025
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



pay = P(−|hasta)·p = 0.1 × 0.5025 = 0.0503; payda = pay + P(−|sağlıklı)·(1 − p) = 0.91 × 0.4975 + pay = 0.5030.

Adım 12 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
▶        pay = (1 - duyarlilik) * p
▶        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
11   return p
```

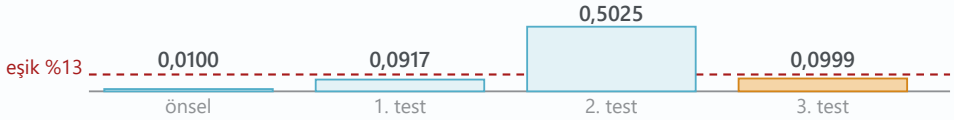
DEĞİŞKENLER

`p = 0.5025` `duyarlilik = 0.9` `yp = 0.09` `sonuc = −` `pay = 0.0503` `payda = 0.5030`

ÇIKTI

```
+ 0.0917
+ 0.5025
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



p = pay / payda = **0.0999**; print ile yazdırılır. Negatif sonuç inancı yeniden eşiğin altına indirdi.

Adım 13 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
▶      p = pay / payda
▶      print(sonuc, round(p, 4))
11   return p
```

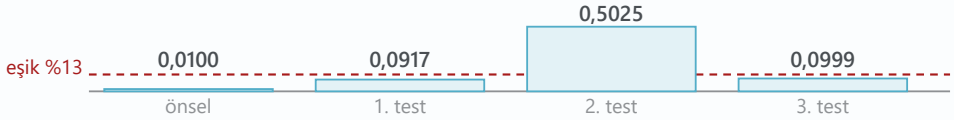
DEĞİŞKENLER

`p = 0.0999` `duyarlilik = 0.9` `yp = 0.09` `sonuc = -` `pay = 0.0503` `payda = 0.5030`

ÇIKTI

```
+ 0.0917
+ 0.5025
- 0.0999
```

Kod İzleme: `bayes(p, duyarlilik, yp, testler)`



Döngü bitti; fonksiyon p = 0.0999 döndürür. Her test, bir öncekinin sonsalını önsel olarak kullanır (ardışık güncelleme).

Adım 14 / 14

```
1  def bayes(p, duyarlilik, yp, testler):
2    for sonuc in testler:
3      if sonuc == "+":
4        pay = duyarlilik * p
5        payda = pay + yp * (1 - p)
6      else:
7        pay = (1 - duyarlilik) * p
8        payda = pay + (1 - yp) * (1 - p)
9      p = pay / payda
10     print(sonuc, round(p, 4))
▶    return p
```

DEĞİŞKENLER

`p = 0.0999` `sonuç = 0.0999`

ÇIKTI

```
+ 0.0917
+ 0.5025
- 0.0999
```

BAYES KURALI

Bayesçi Karar Kuralı: Tedavi mi, Bekle mi?

- Önsel inanç kanıtla güncellenir; sonra her eylemin **beklenen kaybı** sonsal olasılıkla hesaplanır. En düşük beklenen kayıplı eylem seçilir.
- Kayıplar: sağlıklıya gereksiz tedavi 30, hastayı tedavi etmemek 200.
- E[kayıp | tedavi] = (1 − p)·30; E[kayıp | bekle] = p·200. Tedavi ⇔ p > 30/(30 + 200) = **3/23 ≈ %13,0**.

Durum	p	tedavi	bekle	karar
1 pozitif	0,092	27,2	18,3	bekle, yeniden test
2 pozitif	0,503	14,9	100,5	tedavi

Beklenen kayıplar (turuncu = en küçük)

Karar yalnızca olasılığa değil **kayıpların oranına** da bağlıdır: hastalığı kaçırmak ne kadar pahalıysa eşik o kadar düşer.

Karar Ağacı: Ürün Lansmanı ve Pazar Araştırması

- Bir firma yeni ürünü piyasaya **sürüp sürmemeye** karar verecek. Pazar iyi (olasılık 2/5) ise +500, kötü (3/5) ise −200 bin TL; sürmezse 0.
- Önce 20 bin TL'ye **pazar araştırması** yaptırabilir. Rapor kusurludur: P(olumlu | iyi) = 0,8, P(olumlu | kötü) = 0,1.
- Bayes ile: P(olumlu) = 0,4·0,8 + 0,6·0,1 = 0,38; P(iyi | olumlu) = 0,32/0,38 = 16/19; P(iyi | olumsuz) = 0,08/0,62 = 4/31.

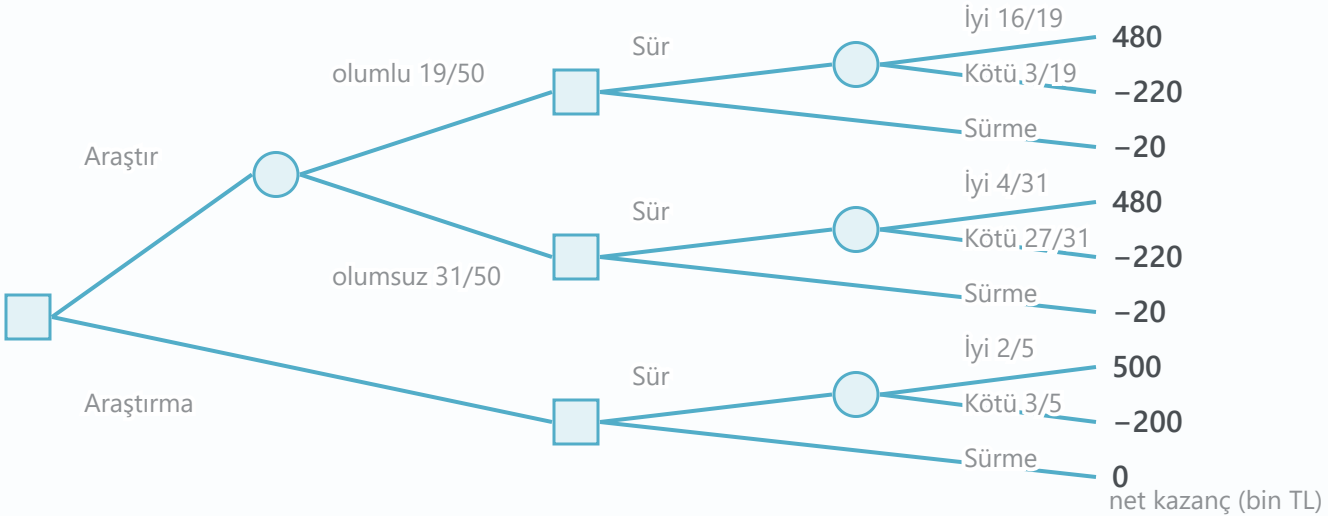
□ Karar düğümü
Karar verici seçer → dallardan en büyük değer.

○ Şans düğümü
Doğa seçer → olasılıklarla ağırlıklı ortalama.

Yaprak
Bir yolun net sonucu.

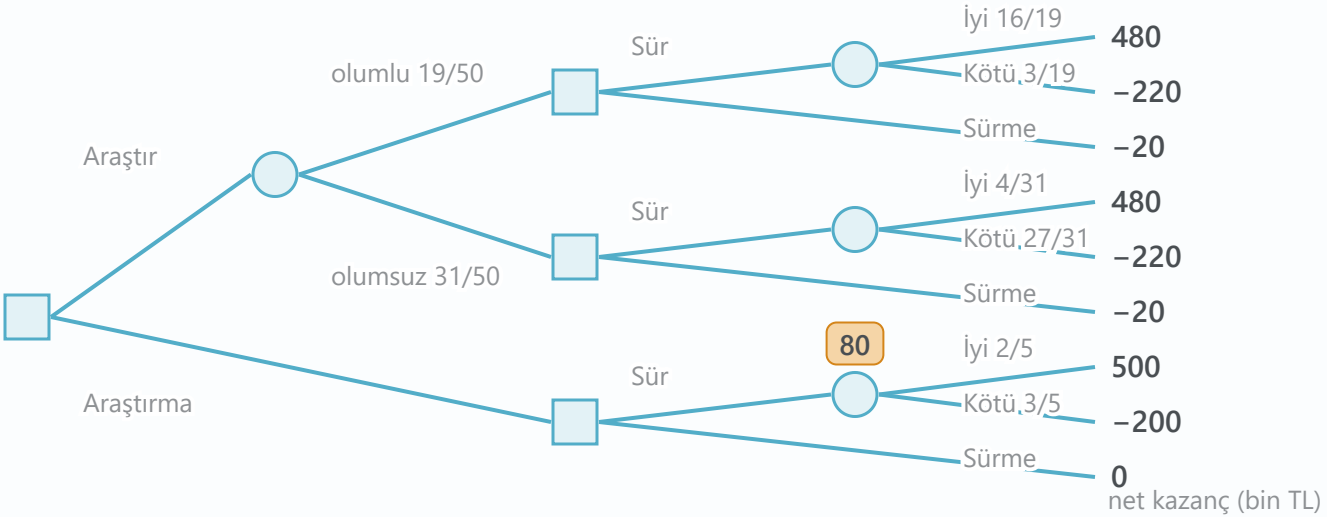
Karar ağacı, doğaya karşı oynanan genişletilmiş formlu bir oyundur; **geriye doğru çıkarım** aynen uygulanır.

Karar Ağacında Geriye Doğru Çıkarım



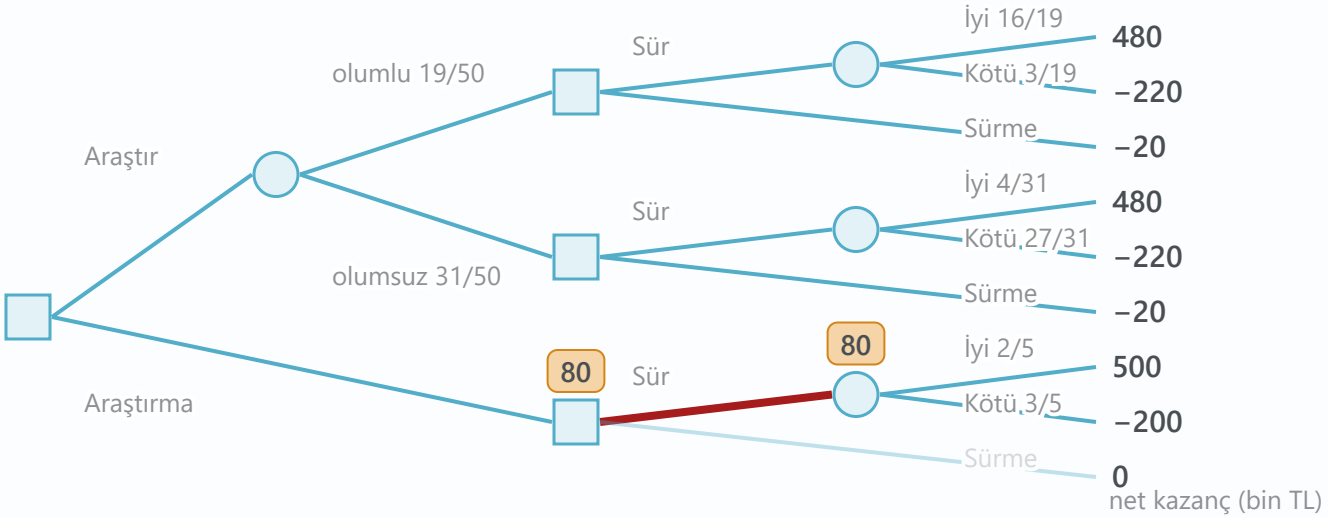
Kare = karar düğümü, daire = şans düğümü. Yapraklarda net kazanç (araştırma yapılırsa 20 düşülmüş). Geriye doğru çıkarım: şans düğümünde **beklenen değer**, karar düğümünde **en büyük** değer.

Karar Ağacında Geriye Doğru Çıkarım



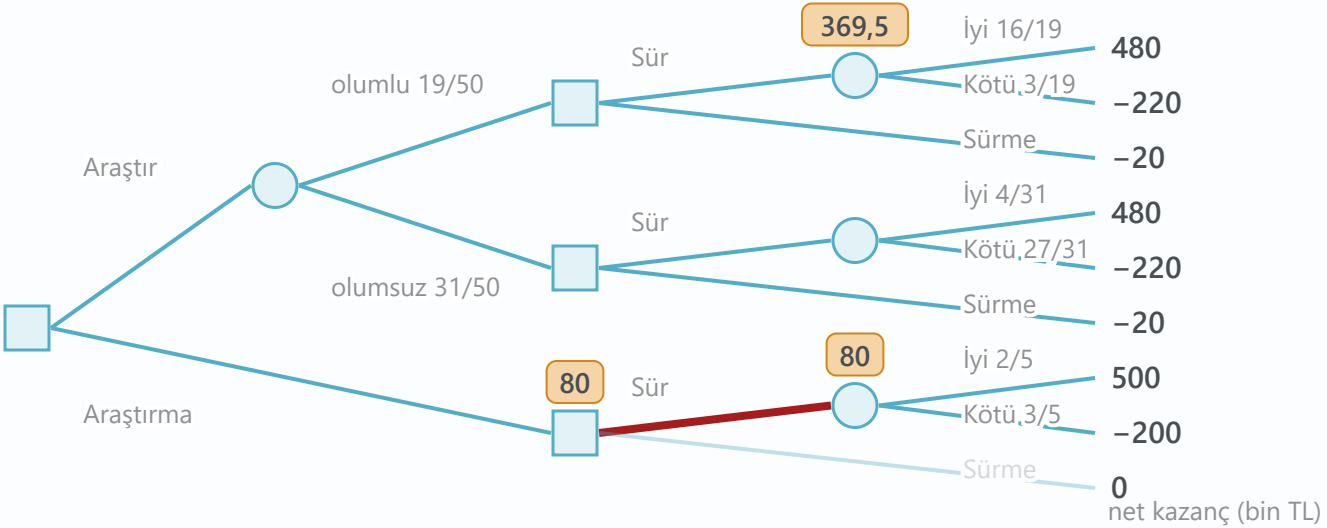
Araştırmasız sürme: 2/5·500 + 3/5·(−200) = 200 − 120 = **80**.

Karar Ağacında Geriye Doğru Çıkarım



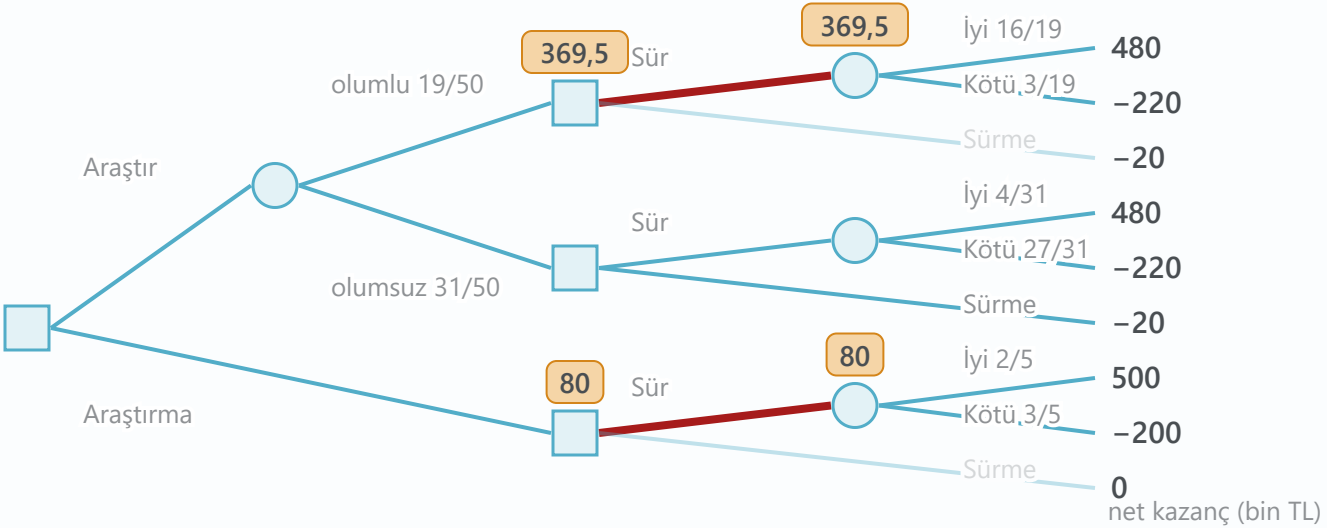
Karar: max(Sür 80, Sürme 0) = 80 ⇒ **Sür**. Sürme dalı elenir.

Karar Ağacında Geriye Doğru Çıkarım



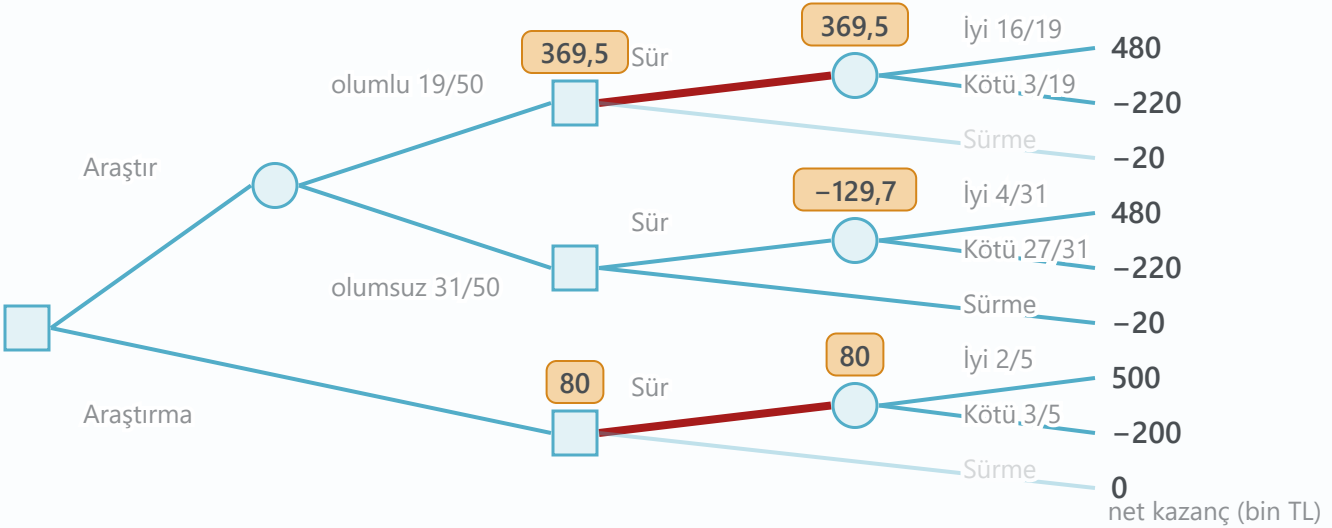
Olumlu raporda P(İyi | olumlu) = 0,32/0,38 = 16/19 (Bayes). Beklenen: 16/19·480 + 3/19·(−220) ≈ **369,5**.

Karar Ağacında Geriye Doğru Çıkarım



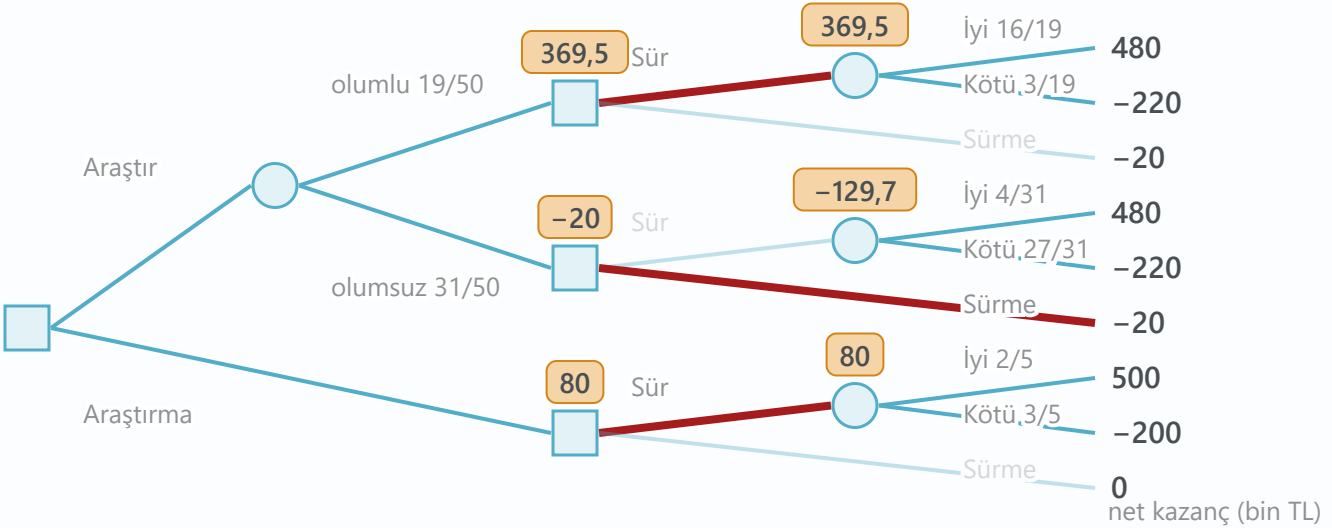
Karar: max(369,5, −20) ⇒ olumlu raporda **Sür**.

Karar Ağacında Geriye Doğru Çıkarım



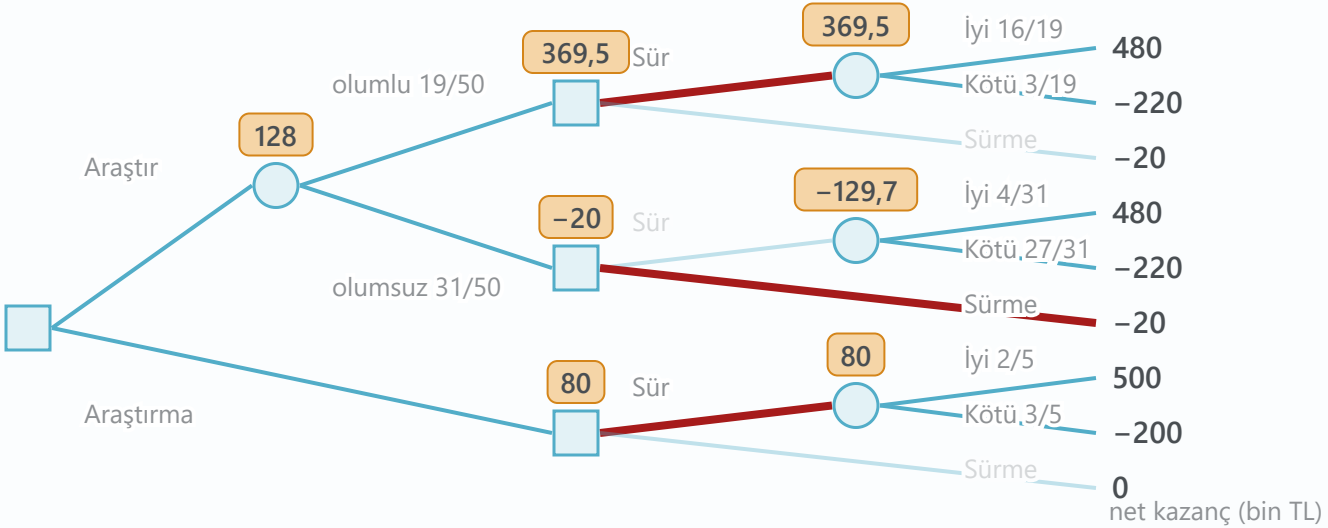
Olumsuz raporda P(İyi | olumsuz) = 0,08/0,62 = 4/31. Beklenen: 4/31·480 + 27/31·(−220) ≈ **−129,7**.

Karar Ağacında Geriye Doğru Çıkarım



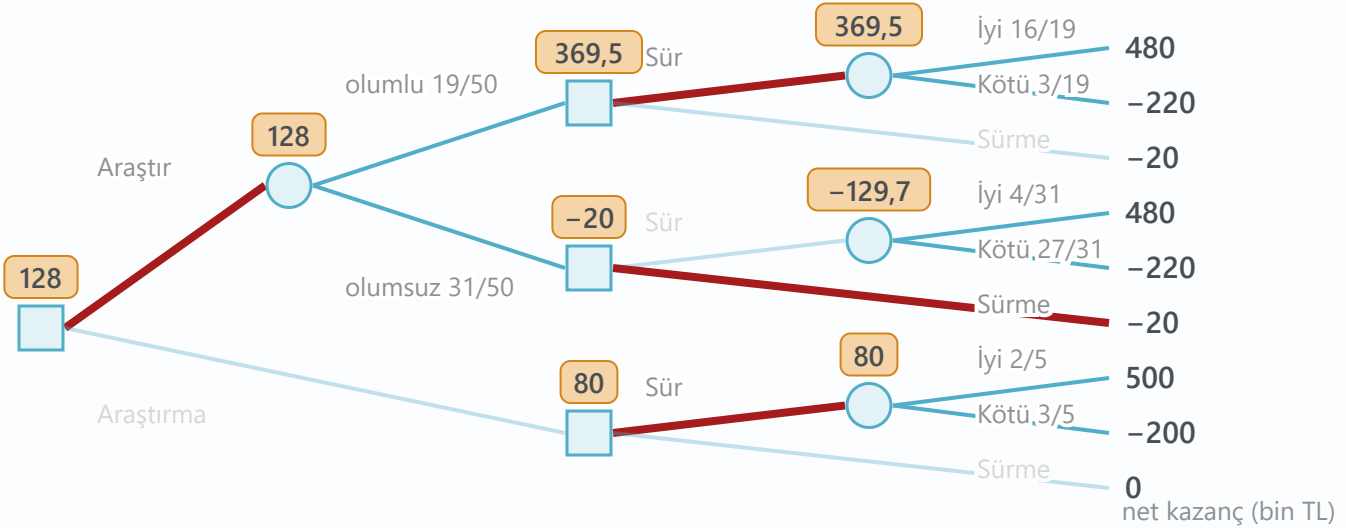
Karar: max(−129,7, −20) = −20 ⇒ olumsuz raporda **Sürme** (yalnızca araştırma maliyeti kaybedilir).

Karar Ağacında Geriye Doğru Çıkarım



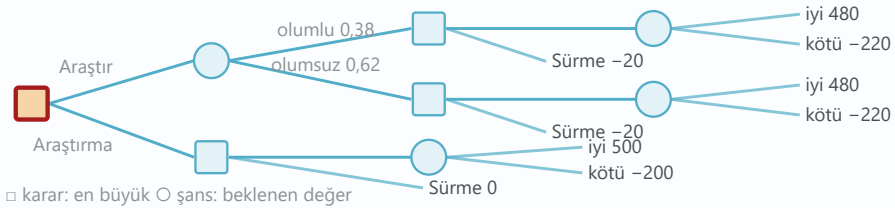
Rapor şans düğümü: 0,38·369,5 + 0,62·(−20) = **128**.

Karar Ağacında Geriye Doğru Çıkarım



Kök: max(Araştır 128, Araştırma 80) ⇒ **Araştır**. Strateji: araştır; olumluysa sür, olumsuzsa sürme.

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(kök)` çağrıldı: karar düğümü; çocukları sırayla değerlendirilir.

Adım 1 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
      tur, dallar = n
      if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
      return max(deger(c) for ad, c in dallar)  # karar
```

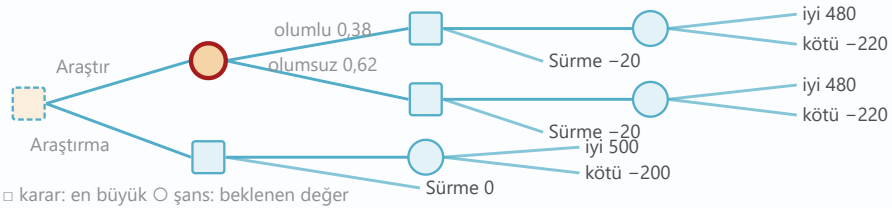
DEĞİŞKENLER

`n = kök`

ÇAĞRI YIĞINI

`deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(Araştır)` çağrıldı: şans düğümü; çocukları sırayla değerlendirilir.

Adım 2 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
▶     tur, dallar = n
▶     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

DEĞİŞKENLER

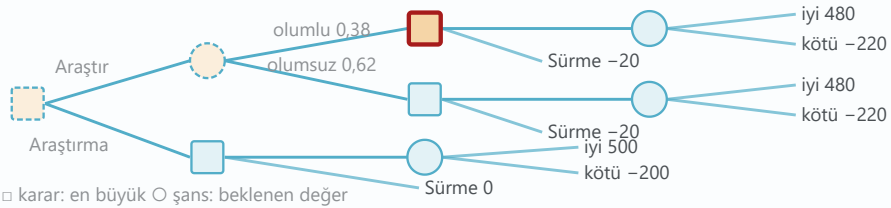
`n = Araştır`

ÇAĞRI YIĞINI

`deger(Araştır)`

`deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(olumlu)` çağrıldı: karar düğümü; çocukları sırayla değerlendirilir.

Adım 3 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
▶     tur, dallar = n
▶     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
▶     return max(deger(c) for ad, c in dallar)  # karar
```

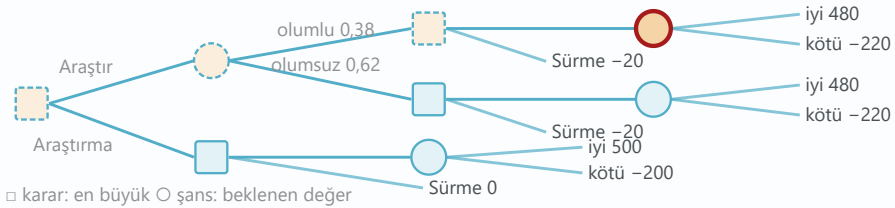
DEĞİŞKENLER

`n = olumlu`

ÇAĞRI YIĞINI

- `deger(olumlu)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(olumlu·Sür)` çağrıldı: şans düğümü; çocukları sırayla değerlendirilir.

Adım 4 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
▶     tur, dallar = n
▶     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

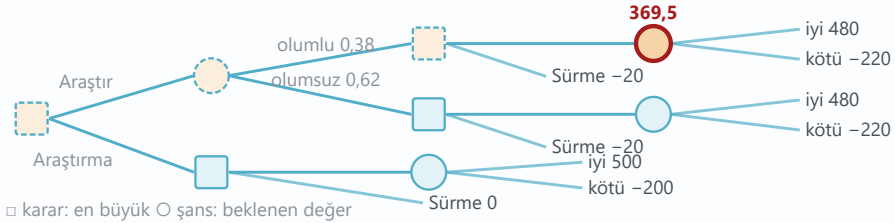
DEĞİŞKENLER

`n = olumlu·Sür`

ÇAĞRI YIĞINI

- `deger(olumlu·Sür)`
- `deger(olumlu)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



olumlu·Sür (şans): 0,842·480 + 0,158·−220 = **369,5**.

Adım 5 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
▶         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

DEĞİŞKENLER

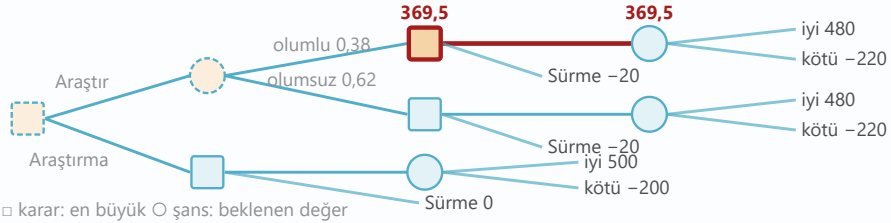
`n = olumlu·Sür`

`return = 369.47`

ÇAĞRI YIĞINI

- `deger(olumlu·Sür)`
- `deger(olumlu)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



olumlu (karar): max(Sür 369,5, Sürme −20,0) = **369,5** → **Sür**.

Adım 6 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
▶     return max(deger(c) for ad, c in dallar)  # karar
```

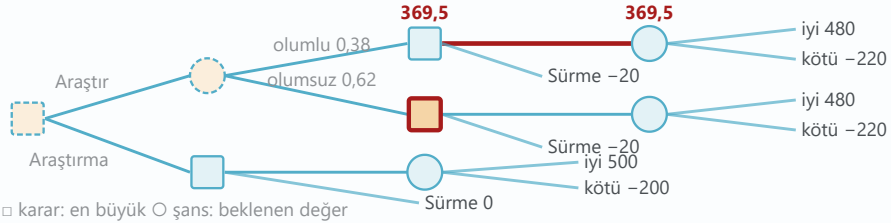
DEĞİŞKENLER

`n = olumlu` `return = 369.47`

ÇAĞRI YIĞINI

- `deger(olumlu)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(olumsuz)` çağrıldı: karar düğümü; çocukları sırayla değerlendirilir.

Adım 7 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
▶     tur, dallar = n
▶     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
▶     return max(deger(c) for ad, c in dallar)  # karar
```

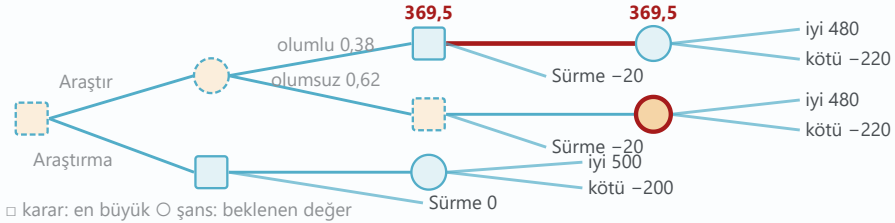
DEĞİŞKENLER

`n = olumsuz`

ÇAĞRI YIĞINI

- `deger(olumsuz)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(olumsuz·Sür)` çağrıldı: şans düğümü; çocukları sırayla değerlendirilir.

Adım 8 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
▶     tur, dallar = n
▶     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

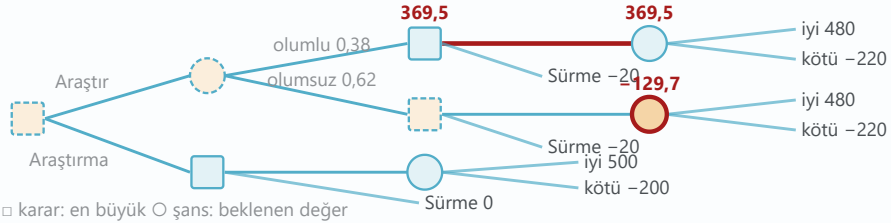
DEĞİŞKENLER

`n = olumsuz·Sür`

ÇAĞRI YIĞINI

- `deger(olumsuz·Sür)`
- `deger(olumsuz)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



olumsuz·Sür (şans): 0,129·480 + 0,871·−220 = **−129,7**.

Adım 9 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
▶         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

DEĞİŞKENLER

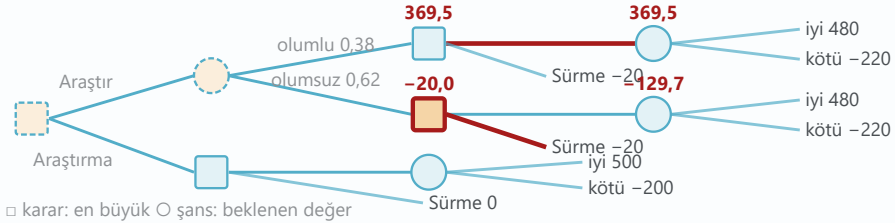
`n = olumsuz·Sür`

`return = -129.68`

ÇAĞRI YIĞINI

- `deger(olumsuz·Sür)`
- `deger(olumsuz)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



olumsuz (karar): max(Sür −129,7, Sürme −20,0) = **−20,0** → **Sürme**.

Adım 10 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
▶     return max(deger(c) for ad, c in dallar)  # karar
```

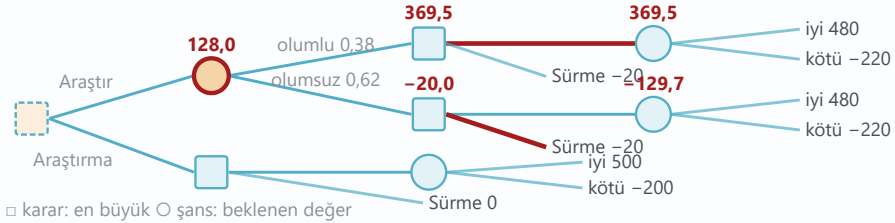
DEĞİŞKENLER

`n = olumsuz` `return = -20`

ÇAĞRI YIĞINI

- `deger(olumsuz)`
- `deger(Araştır)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



Araştır (şans): 0,38·369,5 + 0,62·−20 = **128,0**.

Adım 11 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
▶         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

DEĞİŞKENLER

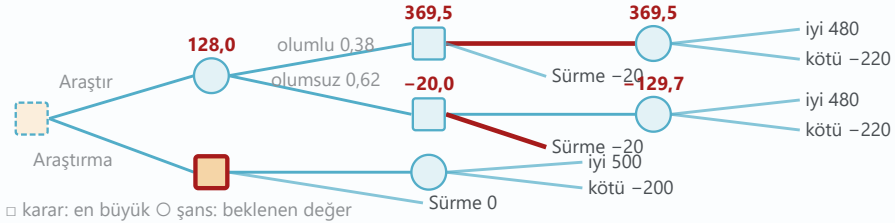
`n = Araştır` `return = 128.0`

ÇAĞRI YIĞINI

`deger(Araştır)`

`deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(Araştırma)` çağrıldı: karar düğümü; çocukları sırayla değerlendirilir.

Adım 12 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
▶     tur, dallar = n
▶     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
▶     return max(deger(c) for ad, c in dallar)  # karar
```

DEĞİŞKENLER

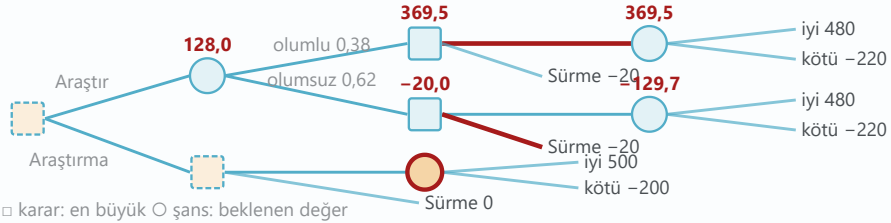
`n = Araştırma`

ÇAĞRI YIĞINI

`deger(Araştırma)`

`deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



`deger(A·Sür)` çağrıldı: şans düğümü; çocukları sırayla değerlendirilir.

Adım 13 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
▶     tur, dallar = n
▶     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

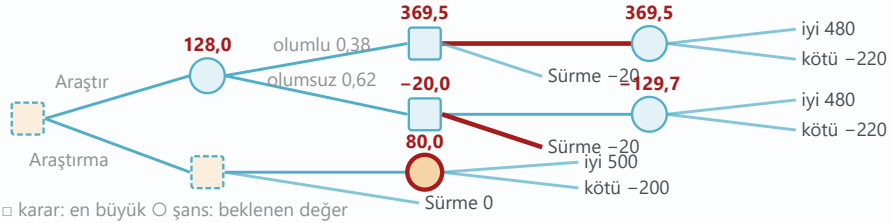
DEĞİŞKENLER

`n = A·Sür`

ÇAĞRI YIĞINI

- `deger(A·Sür)`
- `deger(Araştırma)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



A·Sür (şans): 0,4·500 + 0,6·−200 = **80,0**.

Adım 14 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
▶         return sum(p * deger(c) for p, c in dallar)
7     return max(deger(c) for ad, c in dallar)  # karar
```

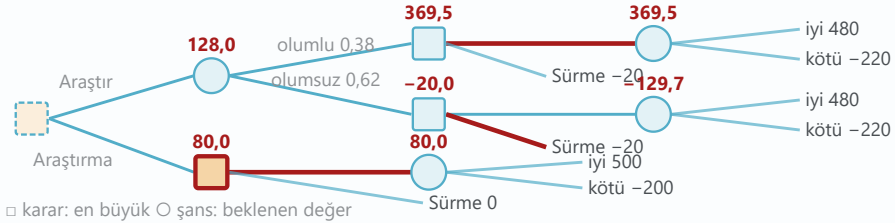
DEĞİŞKENLER

`n = A·Sür` `return = 80.0`

ÇAĞRI YIĞINI

- `deger(A·Sür)`
- `deger(Araştırma)`
- `deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



Araştırma (karar): max(Sür 80,0, Sürme 0,0) = **80,0 → Sür**.

Adım 15 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
▶     return max(deger(c) for ad, c in dallar)  # karar
```

DEĞİŞKENLER

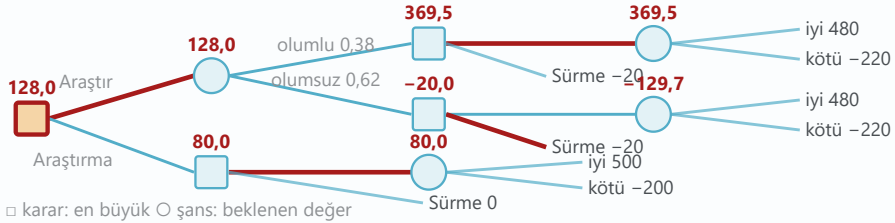
`n = Araştırma` `return = 80.0`

ÇAĞRI YIĞINI

`deger(Araştırma)`

`deger(kök)`

Kod İzleme: Özyinelemeli Geriye Doğru Çıkarım



kök (karar): max(Araştır 128,0, Araştırma 80,0) = **128,0** → **Araştır**. Strateji: araştır; olumluysa sür, olumsuzsa sürme.

Adım 16 / 16

```
1 def deger(n):
2     if not isinstance(n, tuple):    # yaprak
3         return n
4     tur, dallar = n
5     if tur == "S":                  # şans düğümü
6         return sum(p * deger(c) for p, c in dallar)
▶     return max(deger(c) for ad, c in dallar)  # karar
```

DEĞİŞKENLER

`n = kök` `return = 128.0`

ÇAĞRI YIĞINI

`deger(kök)`

Bilginin Beklenen Değeri: EVPI ve EVSI

Durum	Hesap	Değer
Bilgi yok (hemen karar)	Sür: 0,4·500 + 0,6·(−200)	80
.	.	.
.	.	.
.	.	.
.	.	.

Bilgi olmadan en iyi eylem sürmek: beklenen değer 80.

Bilgi ancak kararı değiştirebiliyorsa değerlidir: olumsuz rapor "sürme" kararına yol açtığı için araştırmanın değeri pozitiftir.

Bilginin Beklenen Değeri: EVPI ve EVSI

Durum	Hesap	Değer
Bilgi yok (hemen karar)	Sür: 0,4·500 + 0,6·(−200)	80
Mükemmel bilgi (kâhin)	iyi → sür 500, kötü → sürme 0: 0,4·500 + 0,6·0	200
.	.	.
.	.	.
.	.	.

Bir kâhin pazarın durumunu önceden söyleseydi: iyi pazarda sürer (500), kötüde sürmezdi (0). Beklenen 200.

Bilgi ancak kararı değiştirebiliyorsa değerlidir: olumsuz rapor "sürme" kararına yol açtığı için araştırmanın değeri pozitiftir.

Bilginin Beklenen Değeri: EVPI ve EVSI

Durum	Hesap	Değer
Bilgi yok (hemen karar)	Sür: 0,4·500 + 0,6·(−200)	80
Mükemmel bilgi (kâhin)	iyi → sür 500, kötü → sürme 0: 0,4·500 + 0,6·0	200
EVPI = mükemmel bilginin beklenen değeri	200 − 80	**120**
.	.	.
.	.	.

EVPI = 200 − 80 = **120**: hiçbir bilgi kaynağına bundan fazlası ödenmez (üst sınır).

Bilgi ancak kararı değiştirebiliyorsa değerlidir: olumsuz rapor "sürme" kararına yol açtığı için araştırmanın değeri pozitiftir.

Bilginin Beklenen Değeri: EVPI ve EVSI

Durum	Hesap	Değer
Bilgi yok (hemen karar)	Sür: 0,4·500 + 0,6·(−200)	80
Mükemmel bilgi (kâhin)	iyi → sür 500, kötü → sürme 0: 0,4·500 + 0,6·0	200
EVPI = mükemmel bilginin beklenen değeri	200 − 80	**120**
Örnek bilgi (araştırma, maliyetsiz)	ağaçtaki 128 + araştırma maliyeti 20	148
.	.	.

Kusurlu araştırmanın brüt değeri: ağaçtaki 128'e maliyeti geri ekle ⇒ 148.

Bilgi ancak kararı değiştirebiliyorsa değerlidir: olumsuz rapor "sürme" kararına yol açtığı için araştırmanın değeri pozitiftir.

Bilginin Beklenen Değeri: EVPI ve EVSI

Durum	Hesap	Değer
Bilgi yok (hemen karar)	Sür: 0,4·500 + 0,6·(−200)	80
Mükemmel bilgi (kâhin)	iyi → sür 500, kötü → sürme 0: 0,4·500 + 0,6·0	200
EVPI = mükemmel bilginin beklenen değeri	200 − 80	**120**
Örnek bilgi (araştırma, maliyetsiz)	ağaçtaki 128 + araştırma maliyeti 20	148
EVSI = örnek bilginin beklenen değeri	148 − 80	**68**

EVSI = 148 − 80 = **68** > maliyet 20 ⇒ araştırma yapılmalı (net kazanç 48). Verimlilik EVSI/EVPI = %57.

Bilgi ancak kararı değiştirebiliyorsa değerlidir: olumsuz rapor "sürme" kararına yol açtığı için araştırmanın değeri pozitiftir.

DAVRANIŞSAL KARAR

İnsanlar Her Zaman Rasyonel mi? Sınırlı Rasyonellik

- **Sınırlı rasyonellik** (bounded rationality; Simon): bilgi, zaman ve hesaplama kapasitesi sınırlıdır; insanlar en iyiyi değil "yeterince iyi"yi arar.
- Sapmalar rastgele değil **sistematiktir**: aynı hatalar tekrar tekrar gözlenir, bu yüzden modellenebilir.
- Nedenler: bilgi eksikliği, zaman baskısı, duygular, kısa yollar (sezgisel kurallar).

Aşırı güven

Kendi tahminlerine gereğinden çok güvenmek: yatırımcıların aşırı işlem yapması.

Çerçeveleme etkisi

Aynı seçenek farklı sunulunca kararın değişmesi.

Kayıptan kaçınma

Eşit büyüklükteki kaybın kazançtan daha güçlü hissedilmesi: panik satışı.

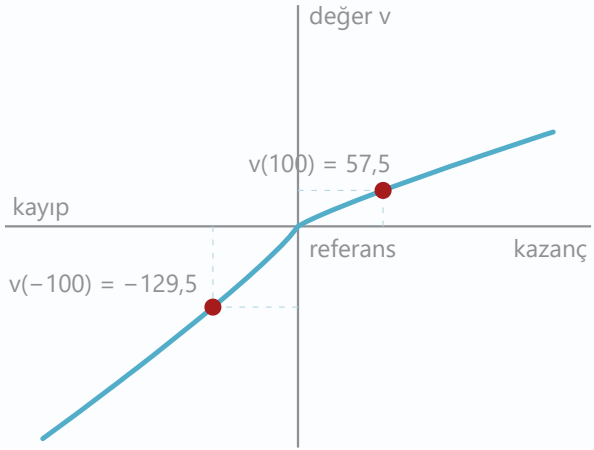
Taban oranı ihmali

Tıbbi test örneğindeki gibi önseli yok saymak.

DAVRANIŞSAL KARAR

Beklenti Kuramı (Prospect Theory)

- Kahneman ve Tversky (1979): sonuçlar toplam servete göre değil, bir **referans noktasına** göre kazanç/kayıp olarak değerlendirilir.
- Değer fonksiyonu kazançta içbükey, kayıpta dışbükey, kayıpta **daha diktir**:
  $v(x) = x^{0,88}$ $(x \geq 0)$; $v(x) = -2,25 \cdot (-x)^{0,88}$ $(x < 0)$
- $\lambda = 2,25$ **kayıptan kaçınma** (loss aversion) katsayısı (Tversky ve Kahneman, 1992 tahmini).
- Sonuç: kazançlarda riskten kaçınan, kayıplarda risk seven davranış.



Referans noktası örneği: 10.000 TL ikramiye bekleyen biri 9.000 TL alınca bunu 1.000 TL'lik bir **kayıp** gibi yaşar; hiç beklemeyen biri için aynı tutar büyük bir kazançtır.

DAVRANIŞSAL KARAR

Kayıptan Kaçınma: Yazı-Tura Bahsi

Bahis
Yazı gelirse +100 TL, tura gelirse −100 TL. Beklenen değer 0.

Beklenti değeri
½·v(100) + ½·v(−100) = ½·57,5 − ½·129,5 = **−36,0** < 0 ⇒ reddedilir.

Kabul eşiği
½·v(G) = ½·2,25·v(100) ⇒ G = $100 \cdot 2{,}25^{1/0{,}88}$ ≈ **251 TL**.

100 TL kaybetme riskini göze almak için yaklaşık **251 TL** kazanç ihtimali istenir. Deneylerde de insanlar tipik olarak kaybın iki katından fazlasını ister.

Olasılık ağırlığı w(p) (γ = 0,61)	**p = 0,01**	**p = 0,50**	**p = 0,99**
Algılanan ağırlık	0,055	0,421	0,912

Küçük olasılıklar abartılır (%1 → ≈ %5,5), büyükler küçümsenir: aynı kişi hem piyango bileti hem sigorta alabilir.

DAVRANIŞSAL KARAR

Çerçeveleme Etkisi: Aynı Seçenek, Farklı Karar

Tversky ve Kahneman (1981): 600 kişiyi etkilemesi beklenen bir salgın için iki program.

Kazanç çerçevesi

+ A: 200 kişi **kurtulur**.
+ B: 1/3 olasılıkla 600 kişi kurtulur, 2/3 olasılıkla kimse kurtulmaz.
+ Katılımcıların **%72**'si A'yı (kesin) seçti.

Kayıp çerçevesi

– C: 400 kişi **ölür**.
– D: 1/3 olasılıkla kimse ölmez, 2/3 olasılıkla 600 kişi ölür.
– Katılımcıların **%78**'i D'yi (riskli) seçti.

A ≡ C ve B ≡ D (beklenen kurtulan hepsinde 200). Beklenen fayda kuramına göre tercih değişmemeli; beklenti kuramı ise bunu kazançta riskten kaçınan, kayıpta risk seven davranışla açıklar.

KOLEKTİF KARARLAR

Kolektif Karar Alma: Bireysel ve Toplumsal Çıkar

- Grup kararlarında **sosyal etki**, uyma baskısı ve ortak çıkarlar rol oynar.
- Topluluk kararları çoğunluk, uzlaşı (konsensüs) ya da oy sistemleriyle alınır.
- Bireysel çıkar kendi faydasını, kolektif çıkar grubun faydasını en büyükler; ikisi çoğu zaman **çelişir**.

Trafik

Herkes en kısa yolu seçer → toplam tıkanıklık artar (Braess).

İş grupları

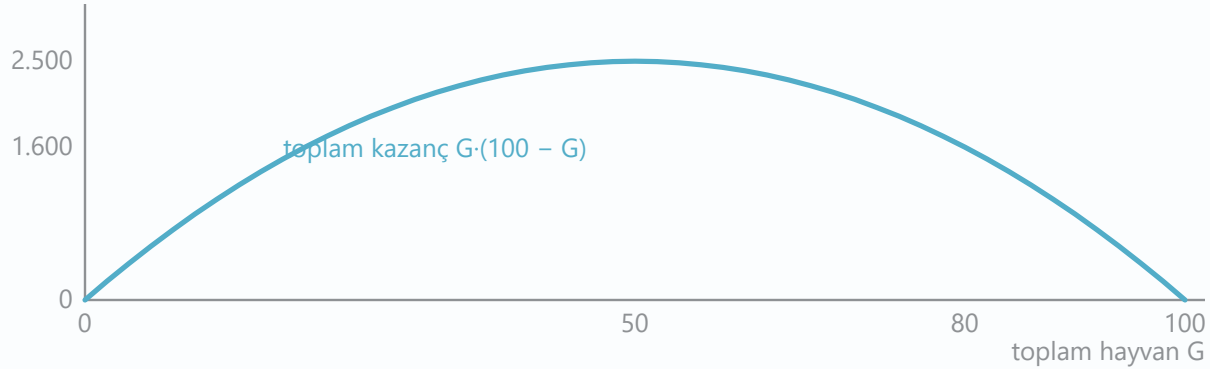
Herkes kendi işine odaklanır → ortak proje aksar (kamu malı).

Ortak kaynak

Balıkçılık, mera, yeraltı suyu, atmosfer → aşırı kullanım.

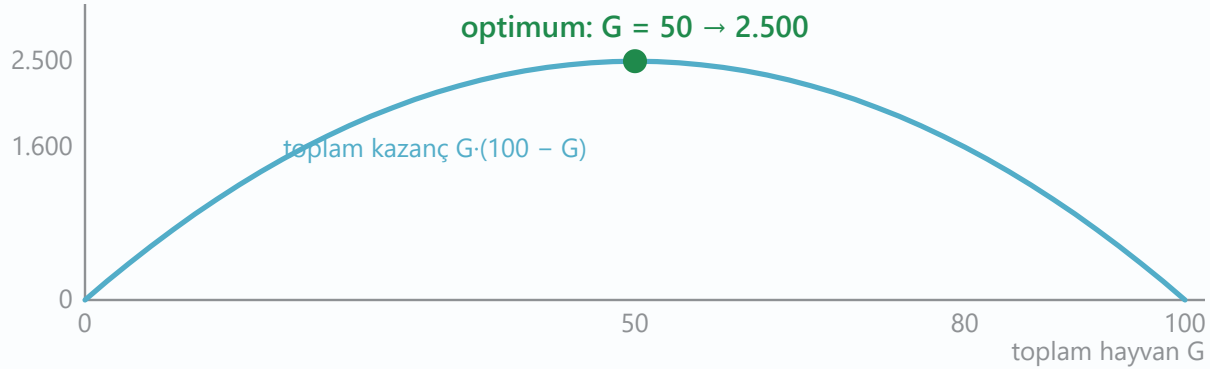
Son örneğin klasik adı: **ortak mallar trajedisi** (tragedy of the commons, Hardin 1968).

Ortak Mallar Trajedisi: Sayısal Model



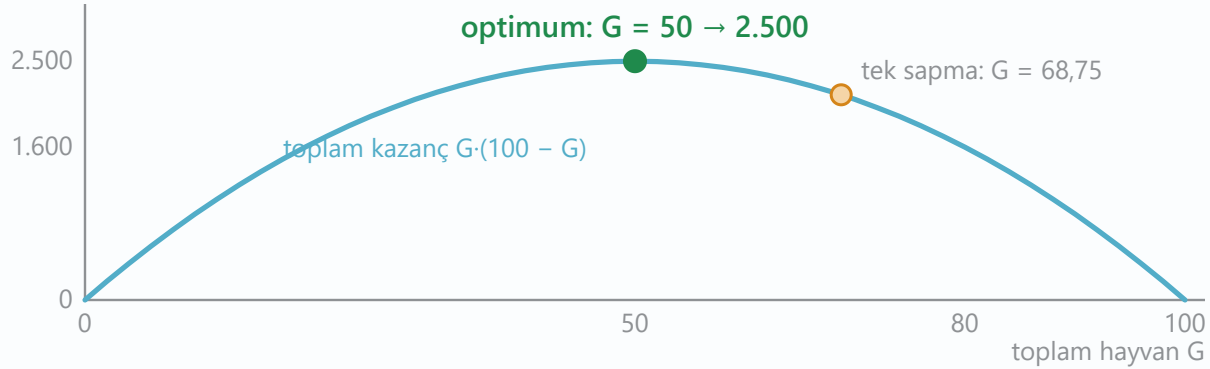
4 çoban ortak meraya hayvan salar. Hayvan başı değer 100 − G (G = toplam hayvan): mera kalabalıklaştıkça her hayvan daha az besi alır. Çoban i'nin kazancı $u_i = g_i(100 - G)$.

Ortak Mallar Trajedisi: Sayısal Model



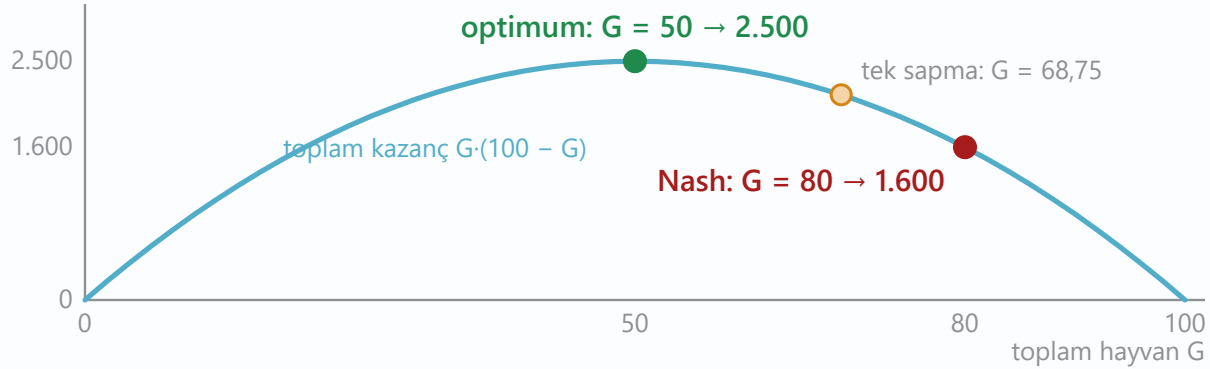
Sosyal optimum: G(100 − G) en büyük ⇒ G = 50, toplam 2.500. Eşit paylaşımda her çoban 12,5 hayvan, kazanç 625.

Ortak Mallar Trajedisi: Sayısal Model



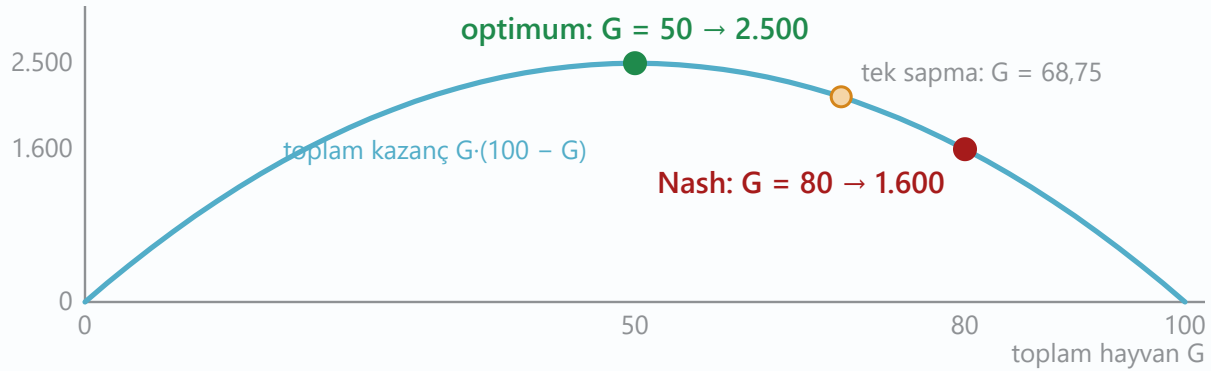
Ama diğerleri 37,5 hayvan salarken çoban 1'in en iyi tepkisi g = (100 − 37,5)/2 = 31,25: kazancı 976,6 > 625. Diğerlerinin kazancı 390,6'ya düşer.

Ortak Mallar Trajedisi: Sayısal Model



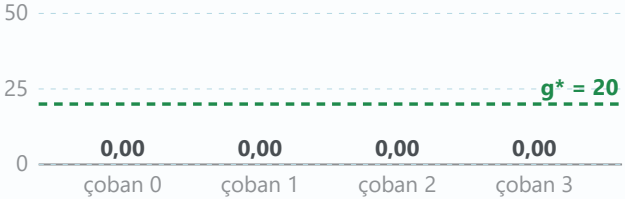
Herkes en iyi tepki verince: g = (100 − 3g)/2 ⇒ **g* = 20**, G = 80. Kontrol: diğerleri 60 iken en iyi tepki (100 − 60)/2 = 20. Kişi başı 400, toplam 1.600.

Ortak Mallar Trajedisi: Sayısal Model



Trajedi: dengede toplam kazanç 1.600, optimumun %64'i. Her çoban kendi hayvanının meraya verdiği zararın yalnızca 1/4'ünü hisseder (dışsallık).

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 0,00 toplam kazanç 0

Çağrı `dinamik(4, 4)` : kimse hayvan salmamış. Çobanlar sırayla, diğerlerinin o anki sayısına en iyi tepki verir: g_i = (100 − diğerleri)/2.

Adım 1 / 18

```
1 def en_iyi_tepki(g, i):
2     digerleri = sum(g) - g[i]
3     return max(0, (100 - digerleri) / 2)
4 def dinamik(n, tur):
▶     g = [0.0] * n
6     for t in range(tur):
7         for i in range(n):          # sırayla güncelle
8             g[i] = en_iyi_tepki(g, i)
9     return g
```

DEĞİŞKENLER

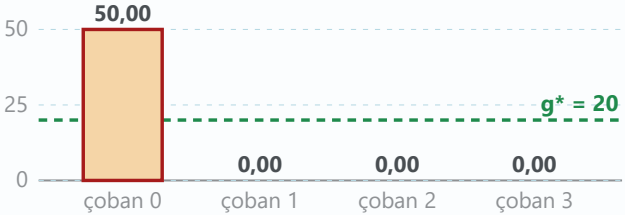
`n = 4` `tur = 4`

`g = [0.0, 0.0, 0.0, 0.0]`

ÇAĞRI YIĞINI

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 50,00 toplam kazanç 2500

Tur 0, çoban 0: diğerleri 0,00 → $g_0 = (100 - 0{,}00)/2 = \mathbf{50{,}00}$. Toplam G = 50,00.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5       g = [0.0] * n
6       for t in range(tur):
▶           for i in range(n):          # sırayla güncelle
▶               g[i] = en_iyi_tepki(g, i)
9       return g
```

DEĞİŞKENLER

`t = 0` `i = 0` `digerleri = 0.0`

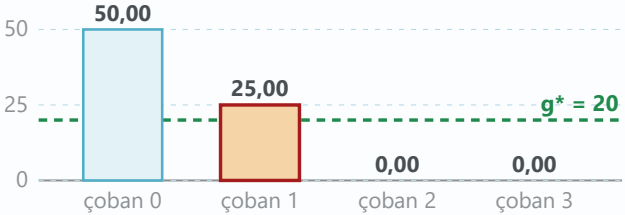
`g = [50.00, 0.00, 0.00, 0.00]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 0)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 75,00 toplam kazanç 1875

Tur 0, çoban 1: diğerleri 50,00 → $g_1 = (100 - 50{,}00)/2 =$ **25,00**. Toplam G = 75,00.


```
1 def en_iyi_tepki(g, i):
      digerleri = sum(g) - g[i]
      return max(0, (100 - digerleri) / 2)
4 def dinamik(n, tur):
5     g = [0.0] * n
6     for t in range(tur):
          for i in range(n):          # sırayla güncelle
              g[i] = en_iyi_tepki(g, i)
9     return g
```

DEĞİŞKENLER

`t = 0` `i = 1` `digerleri = 50.0`

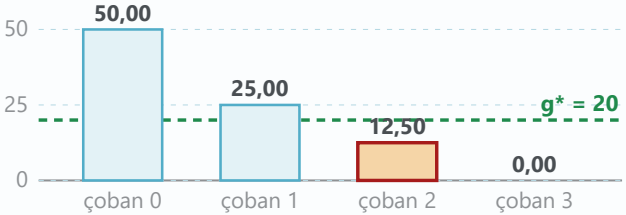
`g = [50.00, 25.00, 0.00, 0.00]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 1)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 87,50 toplam kazanç 1094

Tur 0, çoban 2: diğerleri 75,00 → $g_2 = (100 - 75{,}00)/2 =$ **12,50**. Toplam G = 87,50.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 0` `i = 2` `digerleri = 75.0`

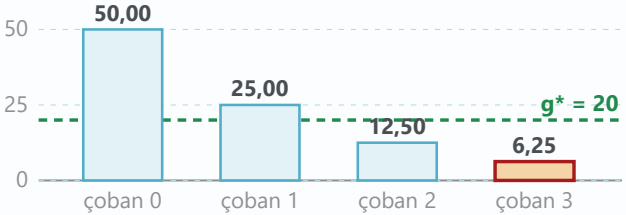
`g = [50.00, 25.00, 12.50, 0.00]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 2)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 93,75 toplam kazanç 586

Tur 0, çoban 3: diğerleri 87,50 → $g_3 = (100 - 87{,}50)/2 =$ **6,25**. Toplam G = 93,75.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 0` `i = 3` `digerleri = 87.5`

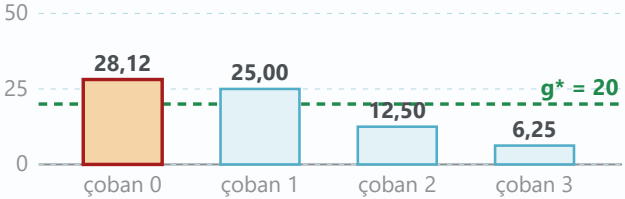
`g = [50.00, 25.00, 12.50, 6.25]`

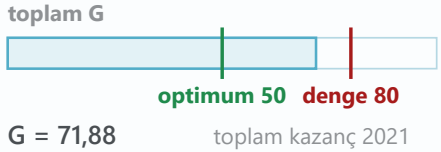
ÇAĞRI YIĞINI

`en_iyi_tepki(g, 3)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





Tur 1, çoban 0: diğerleri 43,75 → $g_0 = (100 - 43{,}75)/2 =$ **28,12**. Toplam G = 71,88.

Adım 6 / 18

```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 1` `i = 0` `digerleri = 43.75`

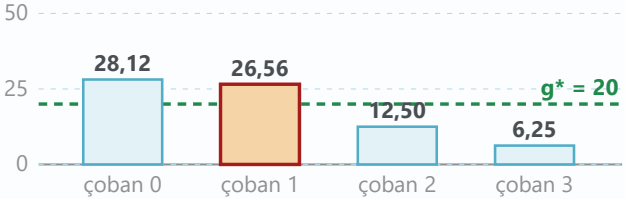
`g = [28.12, 25.00, 12.50, 6.25]`

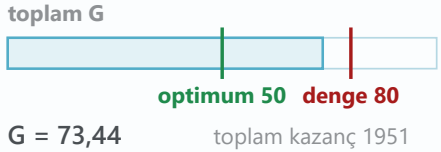
ÇAĞRI YIĞINI

`en_iyi_tepki(g, 0)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





Tur 1, çoban 1: diğerleri 46,88 → $g_1 = (100 - 46{,}88)/2 =$ **26,56**. Toplam G = 73,44.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 1` `i = 1` `digerleri = 46.88`

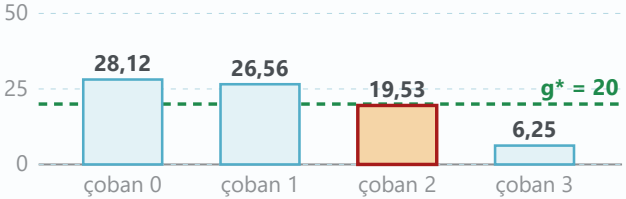
`g = [28.12, 26.56, 12.50, 6.25]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 1)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 80,47 toplam kazanç 1572

Tur 1, çoban 2: diğerleri 60,94 → $g_2 = (100 - 60{,}94)/2 =$ **19,53**. Toplam G = 80,47.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 1` `i = 2` `digerleri = 60.94`

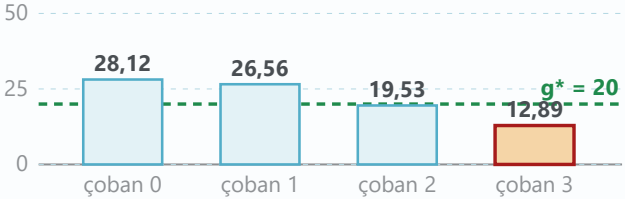
`g = [28.12, 26.56, 19.53, 6.25]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 2)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 87,11 toplam kazanç 1123

Tur 1, çoban 3: diğerleri 74,22 → $g_3 = (100 - 74{,}22)/2 =$ **12,89**. Toplam G = 87,11.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 1` `i = 3` `digerleri = 74.22`

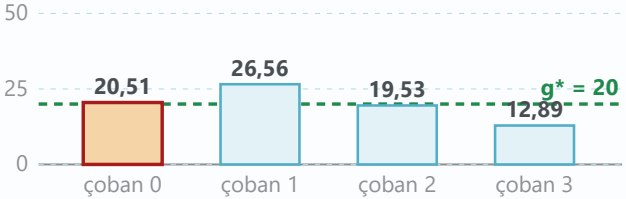
`g = [28.12, 26.56, 19.53, 12.89]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 3)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 79,49 toplam kazanç 1630

Tur 2, çoban 0: diğerleri 58,98 → $g_0 = (100 - 58{,}98)/2 =$ **20,51**. Toplam G = 79,49.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 2` `i = 0` `digerleri = 58.98`

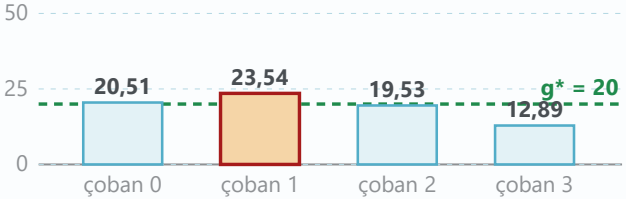
`g = [20.51, 26.56, 19.53, 12.89]`

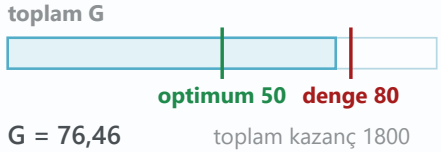
ÇAĞRI YIĞINI

`en_iyi_tepki(g, 0)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





Tur 2, çoban 1: diğerleri 52,93 → $g_1 = (100 - 52{,}93)/2 =$ **23,54**. Toplam G = 76,46.

Adım 11 / 18

```
1  def en_iyi_tepki(g, i):
▶      digerleri = sum(g) - g[i]
▶      return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 2` `i = 1` `digerleri = 52.93`

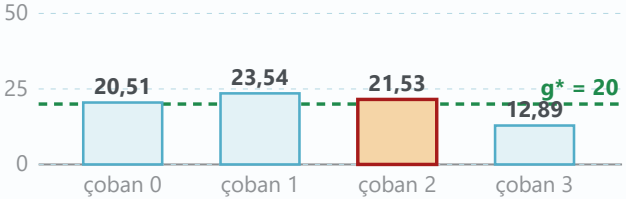
`g = [20.51, 23.54, 19.53, 12.89]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 1)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 78,47 toplam kazanç 1690

Tur 2, çoban 2: diğerleri 56,93 → $g_2 = (100 - 56{,}93)/2 =$ **21,53**. Toplam G = 78,47.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 2` `i = 2` `digerleri = 56.93`

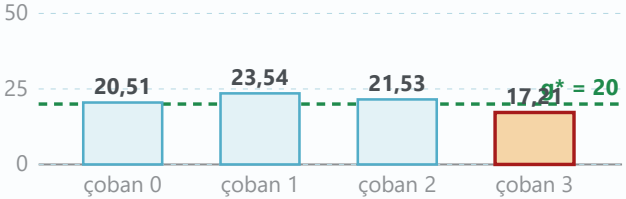
`g = [20.51, 23.54, 21.53, 12.89]`

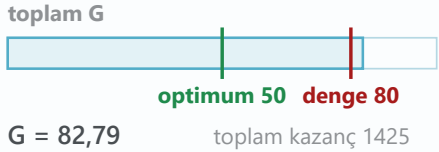
ÇAĞRI YIĞINI

`en_iyi_tepki(g, 2)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





Tur 2, çoban 3: diğerleri 65,58 → $g_3 = (100 - 65{,}58)/2 =$ **17,21**. Toplam G = 82,79.

Adım 13 / 18

```
1  def en_iyi_tepki(g, i):
▶      digerleri = sum(g) - g[i]
▶      return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 2` `i = 3` `digerleri = 65.58`

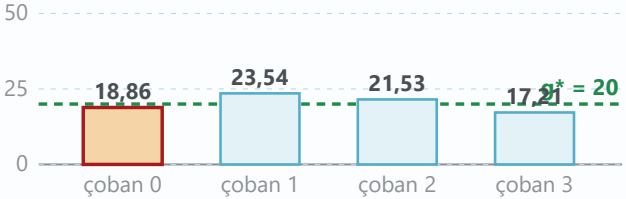
`g = [20.51, 23.54, 21.53, 17.21]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 3)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 81,14 toplam kazanç 1530

Tur 3, çoban 0: diğerleri 62,28 → $g_0 = (100 - 62{,}28)/2 =$ **18,86**. Toplam G = 81,14.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 3` `i = 0` `digerleri = 62.28`

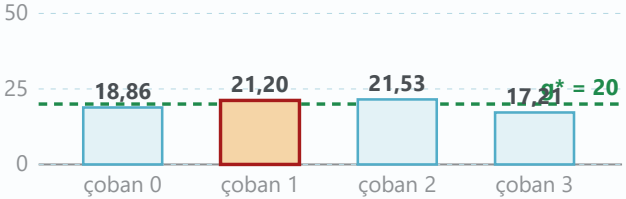
`g = [18.86, 23.54, 21.53, 17.21]`

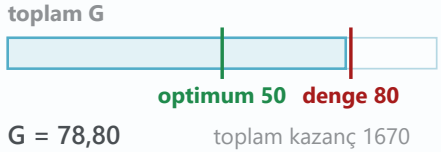
ÇAĞRI YIĞINI

`en_iyi_tepki(g, 0)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





Tur 3, çoban 1: diğerleri 57,60 → $g_1 = (100 - 57{,}60)/2 =$ **21,20**. Toplam G = 78,80.

Adım 15 / 18

```
1  def en_iyi_tepki(g, i):
▶      digerleri = sum(g) - g[i]
▶      return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 3` `i = 1` `digerleri = 57.6`

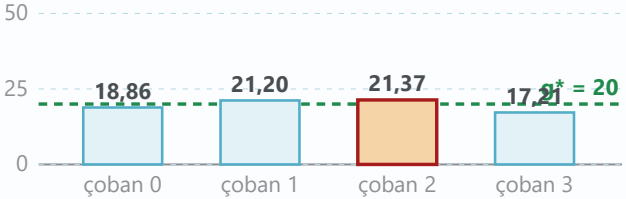
`g = [18.86, 21.20, 21.53, 17.21]`

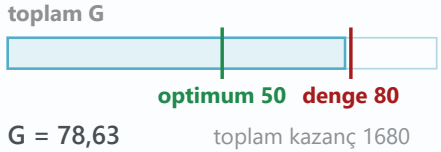
ÇAĞRI YIĞINI

`en_iyi_tepki(g, 1)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





Tur 3, çoban 2: diğerleri 57,27 → $g_2 = (100 - 57{,}27)/2 =$ **21,37**. Toplam G = 78,63.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 3` `i = 2` `digerleri = 57.27`

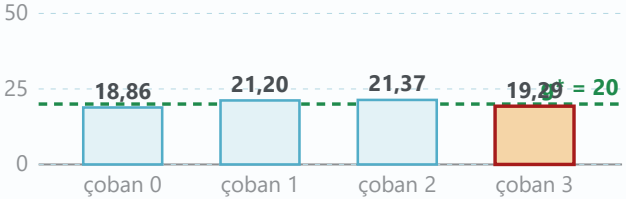
`g = [18.86, 21.20, 21.37, 17.21]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 2)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 80,71 toplam kazanç 1557

Tur 3, çoban 3: diğerleri 61,42 → $g_3 = (100 - 61{,}42)/2 =$ **19,29**. Toplam G = 80,71.


```
1  def en_iyi_tepki(g, i):
▶       digerleri = sum(g) - g[i]
▶       return max(0, (100 - digerleri) / 2)
4  def dinamik(n, tur):
5      g = [0.0] * n
6      for t in range(tur):
▶          for i in range(n):          # sırayla güncelle
▶              g[i] = en_iyi_tepki(g, i)
9      return g
```

DEĞİŞKENLER

`t = 3` `i = 3` `digerleri = 61.42`

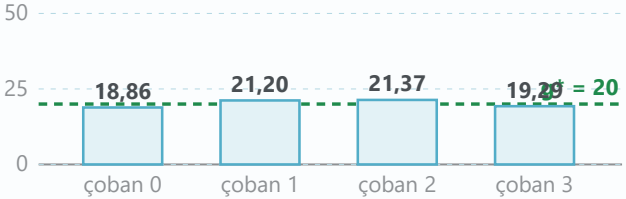
`g = [18.86, 21.20, 21.37, 19.29]`

ÇAĞRI YIĞINI

`en_iyi_tepki(g, 3)`

`dinamik(4, 4)`

Kod İzleme: Çobanlar Dengeyi Nasıl Bulur?





G = 80,71 toplam kazanç 1557

return g: 4 turda G = 80,71. Dinamik Nash dengesine (herkes 20, G = 80) yakınsar (10 turda en büyük sapma 0,018); ama denge sosyal optimumdan (G = 50) uzaktır.

Adım 18 / 18

```
1 def en_iyi_tepki(g, i):
2     digerleri = sum(g) - g[i]
3     return max(0, (100 - digerleri) / 2)
4 def dinamik(n, tur):
5     g = [0.0] * n
6     for t in range(tur):
7         for i in range(n):          # sırayla güncelle
8             g[i] = en_iyi_tepki(g, i)
▶     return g
```

DEĞİŞKENLER

`g = [18.86, 21.20, 21.37, 19.29]`

ÇAĞRI YIĞINI

`dinamik(4, 4)`

Çoban Sayısı Arttıkça ve Çözümler

n	g* (kişi başı)	G*	toplam	optimuma oran
1	50	50	2.500	%100
2	33,33	66,7	2.222	%88,9
4	20	80	1.600	%64
10	9,09	90,9	826	%33,1
100	0,99	99	98	%3,9

Dengede $g^* = 100/(n + 1)$; $n \to \infty$ iken $G^* \to 100$, toplam $\to 0$

Pigou vergisi

Hayvan başı $t = 37{,}5$: denge $g = (100 - t)/5 = 12{,}5 \Rightarrow$ optimum $G = 50$.

Kota / izin

Toplam 50 hayvan hakkı paylaştırılır; ticareti yapılabilir.

Ostrom (2009 Nobel)

Yerel topluluklar izleme, kademeli yaptırım ve ortak kurallarla kaynağı sürdürebilir.

Kolektif Kararı Değerlendirmek: Sosyal Refah Ölçütleri

Politika	Ayşe	Burak	Cem	Σu_i	$\min u_i$	Πu_i
X	9	9	0	18	0	0
Y	5	5	5	15	5	125
Z	8	6	3	17	3	144

Aynı fayda tablosu, üç ölçüt, üç farklı karar (turuncu = sütunun en iyisi)

Faydacı: $\Sigma u_i \rightarrow$ X

Toplamı en büyükler, dağılıma bakmaz (ortak mallar optimumu).

Eşitlikçi (Rawls): $\min u_i \rightarrow$ Y

En kötü durumdakini kayırır: maksiminin toplumsal karşılığı.

Nash sosyal refahı: $\Pi u_i \rightarrow$ Z

Verimlilik ile adalet arasında denge; fayda birimlerinden bağımsız.

Cehalet peçesi: politika, kimin hangi konumda olacağı bilinmeden seçilir. Her konuma eşit olasılık verip beklenen faydayı en büyüklerseniz (Laplace) **faydacı** ölçüte (Harsanyi, 1955), maksimin uygularsanız **eşitlikçi** ölçüte (Rawls, 1971) varırsınız.

ÖZET

Özet

Beklenen fayda

E[u] = Σ p_i u(x_i). Piyango A: E[X] = 450 ama √x ile E[u] = 15 < u(400) = 20.

Risk tutumu

İçbükey → kaçınan (KE 225, prim 225); doğrusal → nötr; dışbükey → seven.

Bayes kuralı

Yaygınlık %1, duyarlılık %90, yanlış pozitif %9 ⇒ P(hasta | +) ≈ %9,2.

Karar ağacı

Şansta ortalama, kararda maksimum. Araştır (128) > araştırmasız (80); EVPI = 120, EVSI = 68.

Beklenti kuramı

Referans noktası, λ ≈ 2,25 kayıptan kaçınma, olasılık ağırlıkları, çerçeveleme.

Ortak mallar

4 çobanlı dengede toplam 1.600, optimum 2.500; vergi, kota ve yerel kurumlar çözüm.

ÖZET

Risk ve Belirsizlik Altında Daha İyi Karar

Bilgi topla, ama değerini hesapla

Güncel veri kararı iyileştirir; EVSI maliyetinden büyükse toplamaya değer.

Senaryo analizi ve karar ağacı

Olası durumları ve olasılıklarını açıkça yaz; geriye doğru çöz.

Önseli unutma

Taban oranlarını Bayes kuralıyla hesaba kat; doğal sıklıklarla düşün.

Rakipleri ve çeşitlendirmeyi düşün

Oyun kuramı ve simülasyon ile rakip davranışını modelle; portföy çeşitlendirmesiyle riski azalt.

ÖZET

Etkileşimli Oyunlar

Bayesçi Karar Laboratuvarı
Önsel, olabilirlik ve kayıplara göre sonsal ve karar.

Blackjack
Kart çek / dur: her hamlede beklenen değer karşılaştırması.

Mayın Tarlası
Açılan sayılara göre hücrelerin mayın olasılığını güncelleme.

Pişti
Oynanan kartları hatırlayarak rakibin elindeki kartlar hakkında çıkarım.

Hokm
Eksik bilgili, takım hâlinde oynanan kart oyunu: koz seçimi ve sinyalleşme.

Mangala
Mükemmel bilgili sıralı oyun: hamle ağacında ileriye bakma.

Poker
Eksik bilgi, olasılık ve blöf: rakibin elini olasılıkla tahmin etme.

Tavla
Zarlı oyun: şans altında beklenen değeri en büyükleyen hamle.

ÖZET

Örnek Problemler

1. Piyango: %20 olasılıkla 2.500 TL, aksi hâlde 100 TL. Beklenen değer? $u(x) = \sqrt{x}$ için kesinlik eşdeğeri ve risk primi?
2. Sigorta örneğinde kayıp olasılığı %20 olsaydı en fazla ödenecek prim ne olurdu?
3. Tıbbi testte yaygınlık %10 olsaydı (duyarlılık %90, yanlış pozitif %9) $P(\text{hasta} \mid +)$ kaç olurdu?
4. Karar ağacında araştırma 70 bin TL olsaydı en iyi strateji ne olurdu?
5. Beklenti kuramı parametreleriyle $+200$ / -100 yazı-tura bahsi kabul edilir mi?
6. Ortak mallar modelinde $n = 9$ çoban için denge G^* ve toplam kazanç? Optimumu sağlayan vergi?

▶ **Cevapları göster**

Kaynaklar

- J. von Neumann, O. Morgenstern, *Theory of Games and Economic Behavior*, Princeton University Press, 1944.
- S. Russell, P. Norvig, *Artificial Intelligence: A Modern Approach*, 4. baskı, Pearson, 2020 (Bölüm 12–16: belirsizlik, Bayes, karar kuramı).
- D. Kahneman, A. Tversky, "Prospect theory: An analysis of decision under risk", *Econometrica* 47(2), 1979.
- A. Tversky, D. Kahneman, "The framing of decisions and the psychology of choice", *Science* 211, 1981; "Advances in prospect theory", *J. Risk and Uncertainty* 5, 1992.
- G. Gigerenzer, U. Hoffrage, "How to improve Bayesian reasoning without instruction: Frequency formats", *Psychological Review* 102(4), 1995.
- G. Hardin, "The tragedy of the commons", *Science* 162, 1968; E. Ostrom, *Governing the Commons*, Cambridge University Press, 1990.
- M. Allais, “Le comportement de l'homme rationnel devant le risque”, *Econometrica* 21, 1953; L. J. Savage, “The theory of statistical decision”, *JASA* 46, 1951.
- H. Raiffa, *Decision Analysis*, Addison-Wesley, 1968.
- J. C. Harsanyi, “Cardinal welfare, individualistic ethics, and interpersonal comparisons of utility”, *J. Political Economy* 63, 1955; J. Rawls, *A Theory of Justice*, Harvard University Press, 1971.
- D. Easley, J. Kleinberg, ***Networks, Crowds, and Markets***, Cambridge University Press, 2010 (Bölüm 16: bilgi çağlayanları ve Bayes kuralı). Araç: **Gambit** (şans düğümlü ağaçları çizip geriye doğru çözer).

Son

Bölüm 13: Karar Alma

Sonraki bölüm: Bölüm 14: Gelecek Yönelimler · Çalışma soruları · Terim sözlüğü